GAME BASED LEARNING
AS A TOOL FOR UNDERSTANDING SORTING ALGORITHMS
ANALYZING EFFECTIVENESS AND ENGAGEMENT

A dissertation
presented to the Faculty of Computing
NSBM Green University
in partial fulfillment of the requirements for the degree of
BSc(Hons.) in Computer Science

by

Suneth Sandaruwan

September , 2025

© [2025] by [Suneth Sandaruwan]

Abstract

GAME BASED LEARNING
AS A TOOL FOR UNDERSTANDING SORTING ALGORITHMS:
ANALYZING EFFECTIVENESS AND ENGAGEMENT

Suneth Sandaruwan

This research explores the effectiveness of a game-based learning approach in teaching sorting algorithms to students. Traditional teaching methods often fail to maintain engagement and simplify the abstract nature of algorithm learning. To address this challenge, a user-friendly web application was developed that allows students to play through six levels of a sorting game. The system was designed to measure learning outcomes by presenting a set of multiple-choice questions (MCQs) before and after gameplay. The pre-test evaluated baseline knowledge, while the post-test, conducted after game completion, measured knowledge improvement related to the specific algorithm chosen and played by each participant.

The study employed a quasi-experimental pre-test/post-test design. Data were collected from participants' pre-test scores, gameplay interactions, completion rates, and post-test scores. Comparative analysis between pre-test and post-test results demonstrated measurable improvements in conceptual understanding of sorting algorithms. The findings suggest that game-based learning not only enhances student engagement but also improves knowledge retention and application of algorithm concepts.

This research highlights the value of integrating interactive and user-centered game design into computer science education. It provides evidence that a thoughtfully designed game-based platform can be an effective educational tool for algorithm learning. Recommendations for future improvements and directions for expanding this approach are also discussed.

Acknowledgement

First and foremost, I would like to express my sincere gratitude to my supervisor, **Ms. Kavishka Rajapaksha**, for her invaluable guidance, continuous support, and constructive feedback throughout the course of this research. Her encouragement and expertise were essential in shaping the direction of this study.

I would also like to extend my appreciation to the **faculty members of NSBM Green University**, whose lectures and resources provided me with the foundational knowledge required to carry out this work.

Special thanks go to all the **participants** who willingly took part in this study by engaging with the web application and completing the tests. Their contribution made the research possible.

Finally, I am deeply grateful to my **family and friends** for their unwavering encouragement, patience, and moral support during this journey. Without their support, this research would not have been completed successfully.

Furthermore, I acknowledge the authors and researchers whose work served as the foundation for my study. I also appreciate the open-source communities and platforms, such as Kaggle and Scikit-Learn, for providing valuable datasets and tools that contributed to the implementation of this project. This research would not have been possible without the collective efforts and support of all the individuals mentioned above.

W P S Sadaruwan,
BSc (Hons) In Computer Science,
NSBM Green University, SriLanka
8th of September 2025

Contents

Chapter 01 – Introduction

1.1 Chapter Overview

This chapter introduces the research study by presenting the background of the problem, the specific issues identified within the domain of sorting algorithms, and the motivation behind undertaking the research. It formulates the research problem and identifies the research gap by analyzing the shortcomings of existing approaches. The chapter also outlines the research question, aim, and objectives that drive this study. Furthermore, it provides a rich picture of the proposed solution, resource requirements, and the project scope. Finally, the chapter concludes with a summary that highlights the major discussion points presented.

1.2 Problem Background

Sorting algorithms are fundamental to computer science and play a critical role in data organization, retrieval, and optimization processes. They form the basis for understanding more advanced computational concepts and are widely used in real-world applications such as databases, search engines, and operating systems. Despite their importance, students often find sorting algorithms difficult to comprehend due to their abstract nature and the requirement for step-by-step logical reasoning.

Traditional teaching methods for algorithms rely heavily on theoretical lectures, textual explanations, and static diagrams. While these methods provide formal definitions and stepwise procedures, they often fail to engage students meaningfully. As a result, students tend to memorize algorithms mechanically rather than internalizing how they function. In the long term, this lack of deep understanding creates a barrier to applying algorithmic concepts in practical problem-solving.

Recent advancements in educational technology highlight the potential of **game-based learning** as a tool for improving engagement and retention. Game-based learning leverages interactivity, competition, and rewards to make learning more enjoyable. In the field of computer science education, such approaches have demonstrated success in teaching programming concepts, logic building, and problem-solving. However, a review of the current landscape reveals that most existing sorting algorithm learning tools lack **comprehensive interactivity** and **user-friendly design**, limiting their effectiveness in facilitating deep learning.

1.3 Problem Statement

Although sorting algorithms are fundamental to computer science, their learning remains challenging for many students due to the abstract and process-driven nature of algorithmic concepts. Existing visualization tools and educational games often fail to fully address this issue, either due to limited interactivity, non-intuitive design, or insufficient integration of gamification principles. As a result, students continue to experience difficulties in developing a strong conceptual understanding of sorting algorithms, leading to poor academic performance and reduced motivation in algorithm-related subjects.

1.3.1 General Problem

At a broader level, the lack of effective, engaging, and interactive learning approaches in computer science education contributes to student disengagement and underperformance. According to recent educational reports, over **40% of computer science undergraduates** struggle with algorithmic problem-solving during their early semesters, which negatively impacts their progression in advanced courses. This suggests that the general problem is not only academic underachievement but also a lack of long-term retention of foundational algorithmic knowledge.

1.3.2 Specific Problem

In the specific domain of sorting algorithms, existing educational tools primarily focus on visualizing algorithm steps through animations. While such tools demonstrate the logic of the algorithms, they do not encourage active participation or provide opportunities for experimentation in a gaming context. Furthermore, many existing sorting games lack a **user-friendly interface**, making it difficult for students to engage with them effectively. This gap highlights the need for a **comprehensive, interactive, and gamified learning tool** that emphasizes usability and promotes conceptual clarity.

Thus, the research gap lies in the absence of a **user-friendly game-based learning tool that combines visualization, interactivity, and gamification principles** to effectively teach sorting algorithms. This study addresses this gap by proposing, designing, and evaluating a game-based learning framework tailored to this need.

1.4 Research Question

The research is guided by the following primary question:

- **How effective is a comprehensive game-based learning tool in teaching and learning students about various sorting algorithms and their applications?**

1.5 Research Motivation

The motivation for this research arises from both academic and personal perspectives. From an academic standpoint, there is a growing need to adopt modern, student-centered approaches to teaching algorithms, as traditional methods are no longer sufficient to maintain student interest or ensure deep understanding. Personally, the researcher experienced challenges in learning sorting algorithms during undergraduate studies, particularly due to missed lectures and reliance on theoretical notes. These difficulties inspired a search for more engaging and interactive ways to learn and teach sorting algorithms.

Furthermore, reviewing existing literature and tools revealed a clear gap in the integration of **user-friendly design** and **gamification strategies** in sorting algorithm education. The researcher's interest in software development, game design, and educational technology further fueled the motivation to design a solution that bridges this gap.

1.6 Research Aim

The aim of this research is to design, develop, and evaluate a **game-based learning tool** that leverages interactivity, gamification, and user-friendly design to enhance student understanding and engagement in learning sorting algorithms.

1.7 Research Objectives

The objectives of this study are as follows:

1. **To analyze** existing visualization and game-based tools used for teaching sorting algorithms.
2. **To design and develop** a user-friendly, interactive game-based learning tool that integrates visualization and gamification.
3. **To evaluate** the effectiveness of the proposed tool in enhancing student learning outcomes and engagement.

1.8 Rich Picture of the Proposed Solution

The proposed solution is a game-based learning tool that provides interactive challenges for various sorting algorithms such as Bubble Sort, Insertion Sort, Selection Sort, Merge Sort, and Quick Sort. Students will interact with the game by solving sorting problems, receiving real-time feedback, and progressing through multiple difficulty levels. Administrators also track student progress to evaluate learning effectiveness.(see figure 1.1)

The rich picture of the system can be visualized as follows:

- **Students** → Interact with the game → Solve sorting challenges → Receive feedback → Improve conceptual understanding.
- **System** → Provides levels, hints, visualizations, and real-time interactivity.
- **Outcome** → Enhanced engagement, improved learning outcomes, and deeper understanding of sorting algorithms.

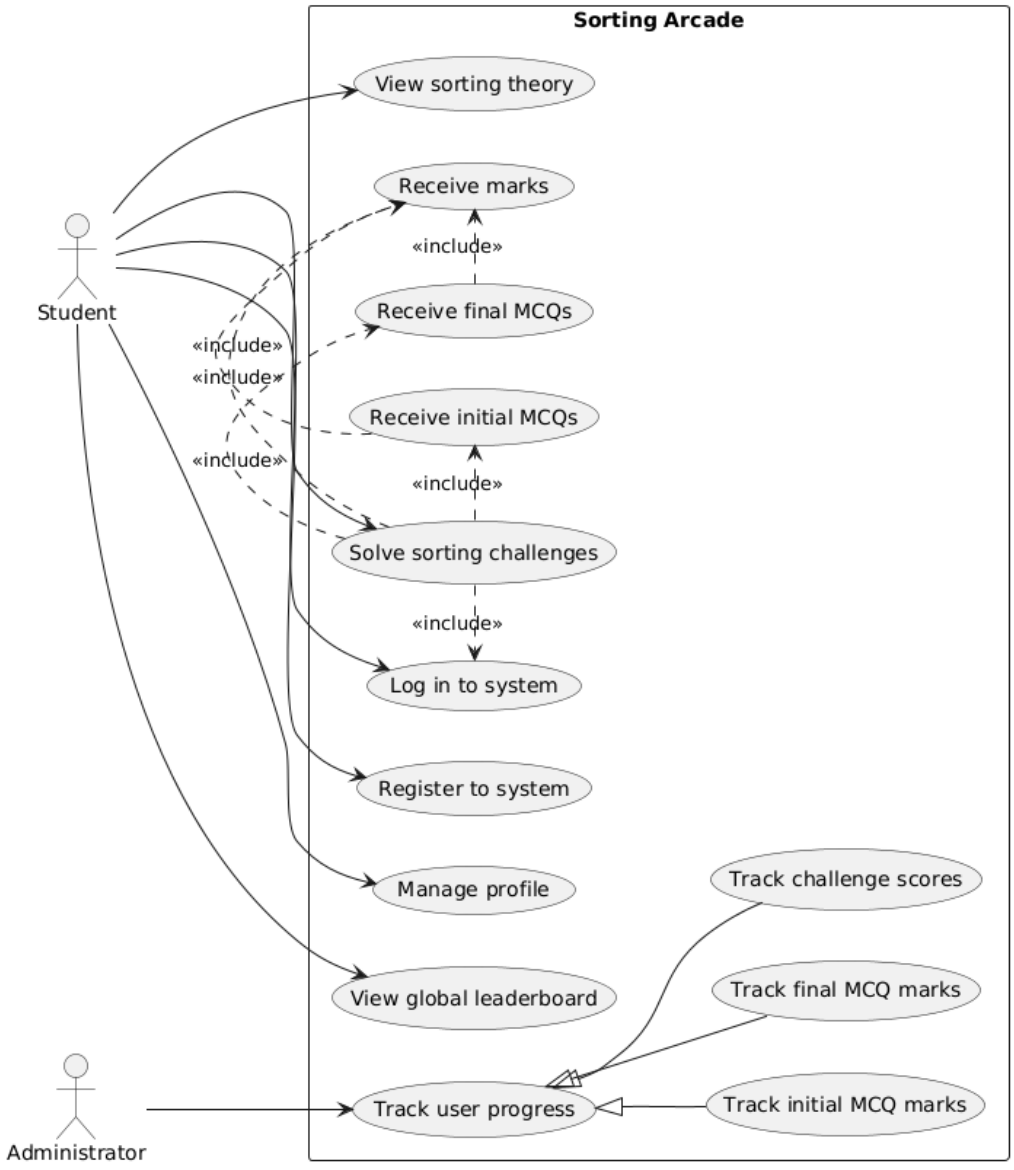


Figure 1.1:user case diagram

1.8.1 Actor and Use Case Analysis

The system is designed around two primary actors with distinct privileges and goals:

1.8.1.1 Student

- The student is the core user of the application, engaged in learning sorting algorithms through an interactive, gamified experience. Their interactions are designed to facilitate a complete learning feedback loop(see fig:1.2)

- Authentication & Profile Management: The journey begins with Registration and Login, securing personalized access. Students can Manage their Profile to control their identity within the system and view own marks for the completed challenges.

- Theory & Context: Before engaging in challenges, students can Study Sorting Theory to understand fundamental concepts. They can also View the Global Leaderboard for social motivation and to gauge their progress against peers.

- Core Learning Loop (Solve Sorting Challenges): This is the central, compound use case of the system. It is a structured process:
    - Pre-Assessment: The student must first complete a set of Initial MCQs to gauge their initial understanding.
    - Interactive Practice: The student then proceeds to Play Interactive Levels. Crucially, the system enhances this experience by providing progressive levels, adaptive hints, data visualizations, and real-time interactivity.
    - Post-Assessment: Upon completing the levels, the student is given a set of Final MCQs to measure learning gain.
- Feedback: Throughout the process, the student will Receive Marks & Feedback on their MCQ answers and challenge performance, closing the learning loop and providing a measure of their achievement.

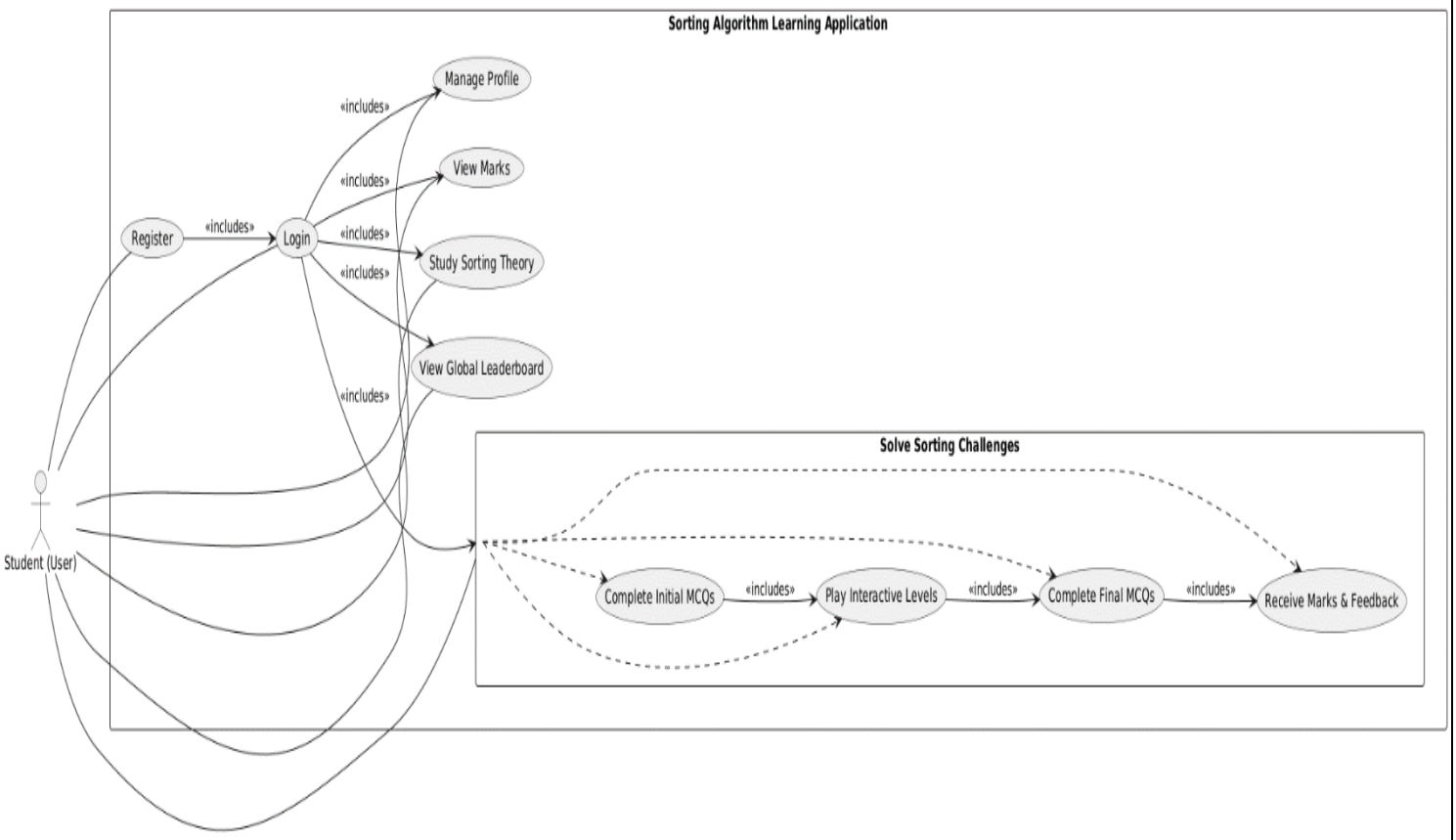


Figure 1.2 user(student) use case diagram

1.8.1.2 Administrator

The Administrator is a supervisory role focused on monitoring and evaluating both individual student progress and overall system effectiveness for research purposes.

Tracking & Analytics: The Administrator can Track User Analytics through a dedicated dashboard. This provides aggregated and individual data on:

- Initial MCQ Metrics to establish a baseline of student knowledge.
- Final MCQ Metrics to measure learning outcomes and knowledge retention.
- Challenge Score Progression to analyze engagement, skill improvement, and the correlation between practical challenge performance and theoretical knowledge.

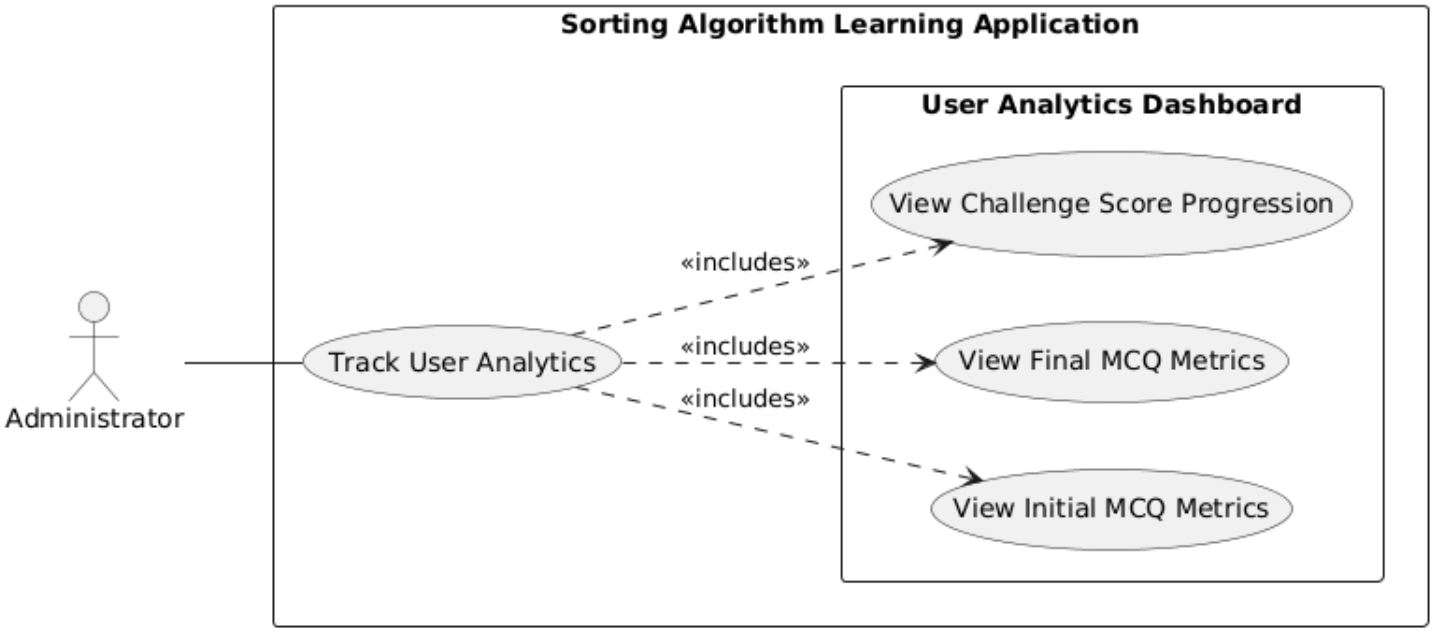


Figure 1.3 Admin use case

1.8.1.3 System Characteristics

The use case diagram (see fig 1.4) highlights key systemic qualities:

- Structured Pedagogy: The enforced sequence of Initial MCQs → Interactive Challenges → Final MCQs creates a controlled experiment environment ideal for research, allowing for clear pre- and post-intervention comparison.
- Gamification: Elements like the Global Leaderboard, Marks, and progressive Levels are designed to increase student motivation and engagement.
- Supportive Learning Environment: The provision of Hints and Visualizations within the challenges scaffolds the learning process, helping students overcome difficulties without excessive frustration.

- Data-Driven Research: The specific, separate analytics functions for the Administrator are explicitly designed to facilitate the extraction of meaningful data for academic research on the effectiveness of the interactive learning tool.
- This rich picture establishes a solid foundation for the system's design, clearly outlining the scope of functionality and the interactions between users and the proposed system.(see fig 1.4)

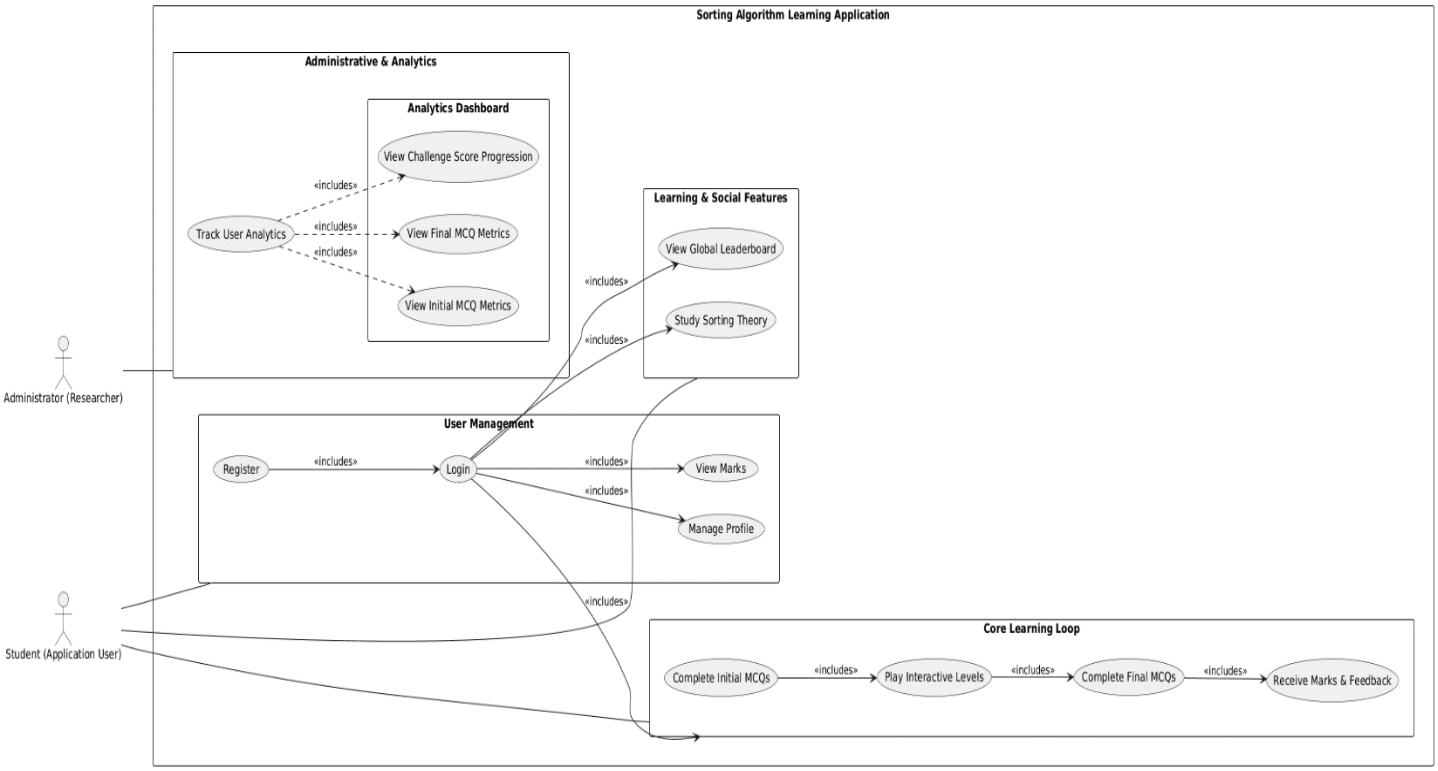


Figure 1.4 overall use case design

1.9 Resource Requirements

1.9.1 Hardware

- Laptop/PC with at least Intel i5 processor (or equivalent).
- Minimum 8GB RAM and 256GB storage.
- Stable internet connection for development and deployment testing.

1.9.2 Software

- **Frontend Development:** React.js
- **Backend Development:** Node.js with Express.js
- **Database:** MySQL
- **IDE:** Visual Studio Code

1.10 Project Scope

In Scope	Out of Scope
Development of a game-based learning tool focusing on sorting algorithms (Bubble Sort, Selection Sort, Insertion Sort, Merge Sort and Quick Sort)	Teaching all algorithm categories such as graph algorithms, dynamic programming, or cryptography
Integration of gamification elements (levels, hints, points, feedback)	Traditional classroom-only teaching methods without digital intervention
Evaluation of the tool with undergraduate computer science students	Large-scale deployment across multiple universities

Table 1.1: project scope table

This ensures the project remains focused on solving the identified problem without becoming too broad or unmanageable.

1.11 Chapter Summary

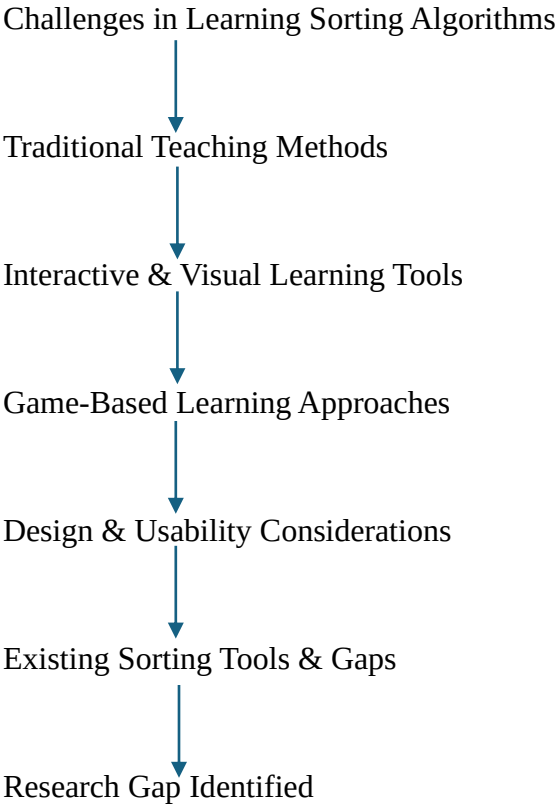
This chapter introduced the research by outlining the problem background, problem statement, and the specific challenges faced by students in learning sorting algorithms. It defined the research question, aim, and objectives that drive this study, along with the motivation behind pursuing the research. The proposed solution was presented through a rich picture, and the resource requirements and project scope were detailed. By narrowing down the scope and identifying a clear research gap, this chapter sets the foundation for the literature review and methodology that follow in subsequent chapters.

Chapter 02 - Literature Review

2.1 Chapter Overview

This chapter reviews literature relevant to sorting algorithm education and game-based learning. It explores the challenges students face, traditional and interactive teaching approaches, the emergence of game-based learning (GBL), design and usability considerations, and existing tools. The analysis concludes by identifying the research gap that motivates this study.

2.2 Conceptual Map of the Literature



2.3 Domain Overview

2.3.1 Challenges in Learning Sorting Algorithms

Sorting algorithms are fundamental yet challenging due to their abstract nature and algorithmic complexity. Lecture-based teaching often fails to illustrate the dynamic processes, resulting in limited understanding and difficulty in applying knowledge [1].

2.3.2 Traditional Teaching Methods

Traditional pedagogical methods rely heavily on passive learning, which does not cater to diverse student needs. As a result, student engagement and retention are often low when studying complex concepts such as sorting [2].

2.3.3 Interactive and Visual Learning Tools

Interactive platforms such as Visualgo provide animations of algorithms, enabling learners to visualize step-by-step execution. These tools improve comprehension by turning abstract processes into concrete visualizations [3].

2.3.4 Game-Based Learning Approaches

GBL integrates algorithms with game mechanics, offering an engaging learning experience. The game Sorted, for instance, demonstrated improved student performance and satisfaction when teaching sorting through gameplay [4].

2.3.5 Design and Usability Considerations

Despite the potential of GBL, poor design or lack of usability reduces its effectiveness. Recent studies emphasize intuitive interfaces, immediate feedback, and adaptability as critical to maximizing learning outcomes [5].

2.4 Existing Systems / Frameworks / Designs

2.4.1 Visualization and Interactive Tools

Visualization tools such as VisuAlgo and AlgoViz present sorting step-by-step. While useful, these are primarily observational and provide limited interactivity [6].

In contrast, Janani [7] developed a card-ranking game-based method where students physically performed sorting actions, leading to improved understanding and positive feedback.

Similarly, 3D immersive learning environments were used by Batsikas and Voyiatzaki [8] to teach sorting algorithms, offering enhanced engagement and conceptual clarity.

2.4.2 Game-Based Initiatives

Meta-analytic reviews confirm that GBL significantly enhances STEM learning outcomes, with medium-to-large effect sizes (Hedges' $g \approx 0.624$), showing measurable improvements over traditional teaching [9].

Al-Khayat et al. [10] also demonstrated that GBL fosters motivation and cognitive development across diverse student groups.

2.4.3 Broader Education Applications

Game-based learning is not limited to algorithms. In programming education, digital game-based methods improved collaboration, engagement, and academic performance compared to lectures [11].

In logistics education, game-based approaches enhanced self-directed learning skills such as initiative, adaptability, and problem-solving [12].

2.4.4 Design and Usability Considerations

Gamification combined with structured instructional design increases usability and engagement in online learning platforms [13].

Adaptive feedback in educational games also supports personalization and retention, enabling better long-term learning outcomes [14].

2.5 Technological Analysis

2.5.1 Algorithmic Analysis

Sorting algorithms differ in complexity. Bubble Sort ($O(n^2)$) is simple but inefficient, Merge Sort ($O(n \log n)$) introduces recursion, and Quick Sort is efficient but conceptually harder. Educational tools must consider these challenges when designing activities [1], [4].

2.5.2 Design Analysis

GBL systems are most effective when integrating progression mechanics, scoring, and adaptability, aligning pedagogy with usability [9], [10].

2.5.3 Workflow Analysis

Most tools follow: Introduction → Visualization → Practice → Assessment. GBL extends this with interactivity and structured feedback loops, reinforcing knowledge through gameplay cycles [12], [14].

2.6 Reflection

The literature shows that:

- Sorting remains difficult due to abstraction.
- Traditional methods are insufficient for deep understanding.
- Visualization tools aid comprehension but lack interactivity.
- GBL offers engagement but requires strong usability and adaptive design.

Research Gap:

No existing tool combines visualization, interactivity, feedback, and user-friendly game mechanics tailored specifically to teaching sorting algorithms. This study addresses that gap.

Chapter 3 – Methodology

3.1 Chapter overview

This chapter explains the research methodology used to design, develop, and evaluate Sorting Arcade. It describes the research paradigm and approach, the research strategy selected, fact-collection mechanisms, and a step-by-step execution workflow that maps to the guideline's required sub-steps. The chapter also presents ethical considerations, limitations, and instructions for where to place screenshots, diagrams, and instruments.

3.2 Research paradigm

This research adopts a pragmatic paradigm. Pragmatism suits applied design research because it prioritizes solutions that work in practice and supports the use of both qualitative and quantitative evidence to evaluate effectiveness. Sorting Arcade is an engineered artefact (a learning application) and thus requires Quantitative evidence (pre/post tests, task completion times, error rates) to measure learning performance.

Using pragmatism allows the study to select methods that best answer the research question (i.e., How effective is the game-based tool?), rather than being constrained by a single epistemological stance.

3.3 Research strategy

The strategy is Design & Development Research (DDR) with an evaluation phase. DDR is appropriate because the primary output is a working educational artefact (Sorting Arcade). DDR stages implemented here are:

1. Problem analysis — identify learning challenges from literature and early user feedback.
2. Design — create UI/UX, game mechanics, database schema, and API contracts.
3. Development — implement front-end (React + Tailwind), backend API (Node/Express), and database (MySQL).
4. Evaluation — usability testing, learning outcome measurement, and system logs analysis.
5. Reflection & iteration — refine the tool based on results.

The evaluation uses both controlled pre/post measurements and in-situ usage data collected during actual play sessions.

3.4 Fact collection mechanisms (data collection)

The validity and reliability of any research study depend largely on the mechanisms employed to collect data. In this research, two complementary data collection approaches were used: secondary data collection and primary data collection.

3.4.1 Secondary Data Collection

Secondary data was collected to establish the theoretical foundation of the study, justify the problem background, and identify the research gap. Scholarly journal articles, conference proceedings, textbooks, and reputable online resources were systematically reviewed. In particular, platforms such as Visualgo and other algorithm visualization websites were examined to understand the strengths and shortcomings of existing approaches. Additionally, prior research studies on game-based learning, sorting algorithm pedagogy, and educational technology adoption were analyzed. This process provided insights into the challenges faced by students when learning sorting algorithms and highlighted the need for innovative and interactive teaching solutions.

3.4.2 Primary Data Collection

Primary data was collected directly from the Sorting Arcade platform. The system was designed to automatically record learners' interactions and assessment results. Each participant attempting a sorting algorithm challenge was presented with two assessment stages:

- Pre-Test (Initial MCQs): A set of five randomly selected questions from the database aimed at measuring learners' prior understanding of the overall sorting algorithm.
- Post-Test (Final MCQs): A set of ten algorithm-specific questions presented after the completion of the challenge levels to assess knowledge gained.

The scores from both assessments were stored in the backend database and later extracted for analysis. By comparing pre-test and post-test results, this study was able to evaluate the educational effectiveness of the Sorting Arcade system.

3.4.3 Justification of Data Collection Approach

The combination of secondary and primary data collection ensured both conceptual grounding and empirical validation. While secondary data provided context and justification for the research gap, primary data allowed for direct measurement of the system's impact on learners. This dual approach strengthened the overall research methodology and provided robust evidence to support the findings.

3.5 Research methodology execution workflow

This section maps the guideline's required sub-steps to concrete actions completed for Sorting Arcade.

3.5.1 Problem Identification

Collated evidence from literature (Chapter 2) and instructor feedback: students struggle with algorithm visualization and active practice.

3.5.2 Relevance Justification

Justified by literature showing GBL effectiveness in CS/STEM and by documented low retention/engagement in algorithm courses.

3.5.3 Comparative analysis and gap justification

Compared visualizers (VisuAlgo), prior GBL prototypes — found limitations in interactivity, usability, and assessment integration. This gap motivated the comprehensive, assessment-integrated Sorting Arcade.

3.5.4 Define and finalize objectives

Objectives finalized (see Chapter 1): identify challenges, analyze existing tools, design & implement Sorting Arcade, and evaluate its effectiveness.

3.5.5 Design / Development / Data management and handling

Design artifacts produced:

- High-level system architecture (frontend ↔ REST API ↔ MySQL).(see chapter 4, fig 4.1)
- UI wireframes and component design (React + Tailwind).
- Database ER diagram (tables: users, questions, quiz_marks, challenge_scores, algorithms etc.).(see chapter 4,fig 4.2)
- API specification (endpoints for auth, questions, scoring).

Development process:

- Continuous deployment: Frontend deployed to Netlify; backend to Google Cloud (with environment variables secured).
- Data handling: questions and scores stored in sorting-arcade-db.

3.5.6 Evaluation and communication

Results will be documented in Chapter 6 (Testing & Evaluation) with tables and figures. Code and anonymized dataset can be archived in a supplementary repository. Findings will be presented to the supervisor and may be used for conference/journal submissions.

3.6 Ethical considerations

- Anonymity & confidentiality: Store participant identifiers separately from analysis data; publish only anonymized results.
- Data protection: Use password protected. cloud storage. (Google cloud console) all user data have encrypted.
- Risk assessment: No physical or psychological risk beyond normal computer use. Provide contact info for questions/complaints.

3.7 Limitations and delimitations

Delimitations (choices made):

- Study focuses only on five sorting algorithms and one web platform (desktop + mobile responsive).
- Participant pool limited to undergraduate CS students, Advanced level students (no high-school learners or professional developers).

Limitations (threats to validity):

- External validity: Results may not generalize to broader populations or different cultural contexts.
- Selection bias: Convenience sampling may attract motivated students (self-selection).
- Technical variability: Network latency (Netlify/Cloud) could affect timing measures mitigate by recording both client timestamps and server-side logs.
- Learning transfer: Short-term post-tests measure immediate gains; long-term retention not assessed unless follow-up testing is added.

3.8 Operationalization mapping objectives to measurable variables

Research objective	Indicator(s)	Instrument	Analysis
Identify student challenges	Reported difficulties, baseline MCQ scores	Pre-test MCQs, interviews	Descriptive stats; thematic analysis
Analyze existing tools	Feature checklist, comparative ratings	Literature review & heuristic evaluation	Comparative matrix
Design & implement tool	Working prototype, UI metrics	Development artifacts, logs	Qualitative evaluation; functional testing
Evaluate effectiveness	Learning gain, accuracy, usability	Pre/post MCQs, challenge scores	Paired tests, effect size, thematic analysis

Table 3.1:mapping objective to measurable variables

3.9 Chapter summary

This chapter realigned the methodology. It adopted a pragmatic, Design & Development Research strategy. Data collection methods (pre/post MCQs, system logs) and ethical procedures were described in detail.

Chapter 4 – System Design and Implementation

4.1 Introduction

This chapter describes the design and implementation of the Sorting Arcade system, which was developed as a game-based learning tool to enhance students' understanding of sorting algorithms. The chapter provides a comprehensive overview of the system architecture, database design, user interface structure, core functional modules, assessment mechanisms, and deployment strategy. Furthermore, it explains the detailed working principles of each algorithm-specific module, the integration of multiple-choice questions (MCQs), and the mechanisms for scoring, leaderboard management, and administrative oversight. By elaborating on both design and implementation aspects, this chapter demonstrates how methodological planning was translated into a functional and scalable educational system.

4.2 System Architecture

The Sorting Arcade application was designed following a three-tier architecture (see figure 4.1) comprising the presentation layer, application layer, and data layer.

Presentation Layer (Frontend):

Developed using React.js and Tailwind CSS, this layer handles the user interface and interaction logic. The frontend is responsible for rendering algorithm visualizations, challenge gameplay, MCQ interfaces, and leaderboard displays. It was hosted on Netlify.

Application Layer (Backend):

Implemented using Node.js with the Express.js framework, the backend manages application logic and communication between the frontend and database. It provides a set of RESTful APIs that handle user authentication, question retrieval, challenge result submission, and score calculation. The backend was deployed on Google Cloud Console for scalability and reliability.

Data Layer (Database):

A cloud-hosted (Google Cloud SQL) MySQL database stores user credentials, MCQs, challenge attempts, and score records. Data integrity and referential consistency are maintained through primary key–foreign key relationships.

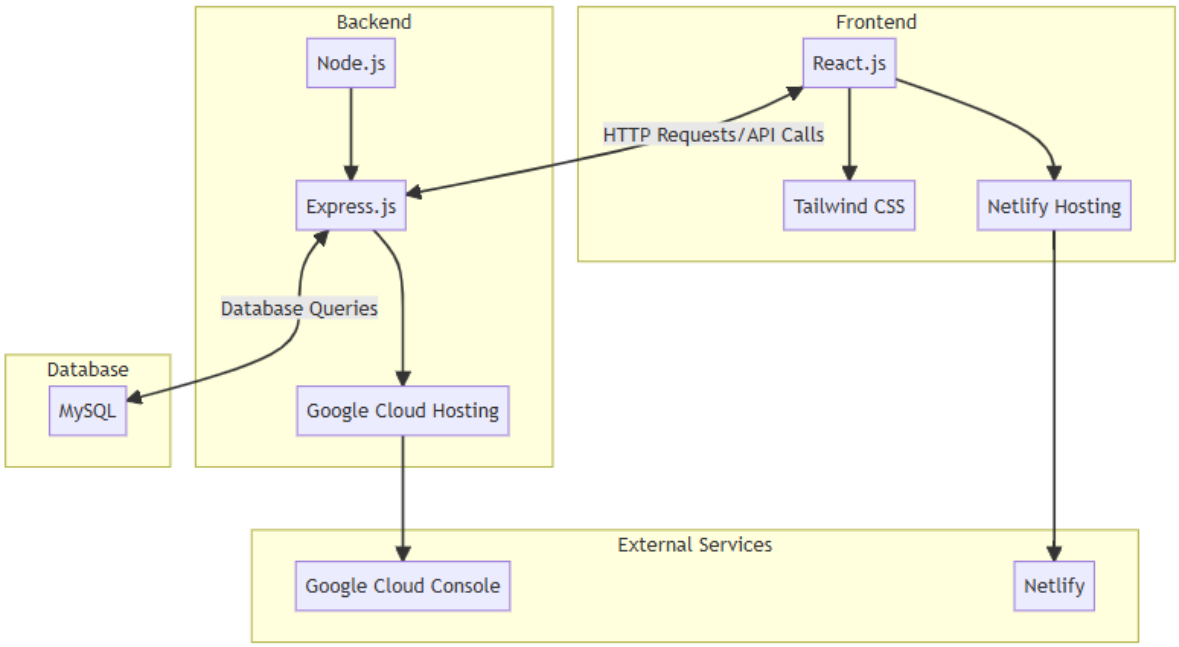


Figure 4.1: system architecture diagram

In operation, the architecture follows this flow: a user interacts with the frontend; the request is sent to the backend API; the backend processes the request and queries the database; finally, results are returned to the frontend for display. This separation of concerns ensures modularity, maintainability, and scalability of the system.

4.3 Database Design

The database, named sorting-arcade-db, was designed with normalization principles to ensure efficient storage and retrieval of data.

The key tables are:

1. Users Table (users): Stores user profile data including user_id, username, email, password_hash, etc.Passwords are securely stored using hashing techniques.

2. Questions Table (questions): Contains MCQs for different sorting algorithms, categorized by algorithm type (e.g., bubble, insertion) and difficulty level (initial or final).

3. Scores Table (algorithm_scores): Records final MCQ results for each user. Each entry is linked to the user_id and algorithm_id.

4. Algorithm table (algorithm) : Contains algorithm names and ids for each algorithm.

5. Chanalge Socore(challenge_scores) : Record algorithm challenge scores played by users. It connected with user table and algorithms table via user_id and algorithm_id.

6. Quiz attempts record (quiz_marks,quiz_attempts) : Record initial and final MCQ quiz attempted by user and record faced questions and provided answers by user, correct answer count stores in challenge_scores table as well.

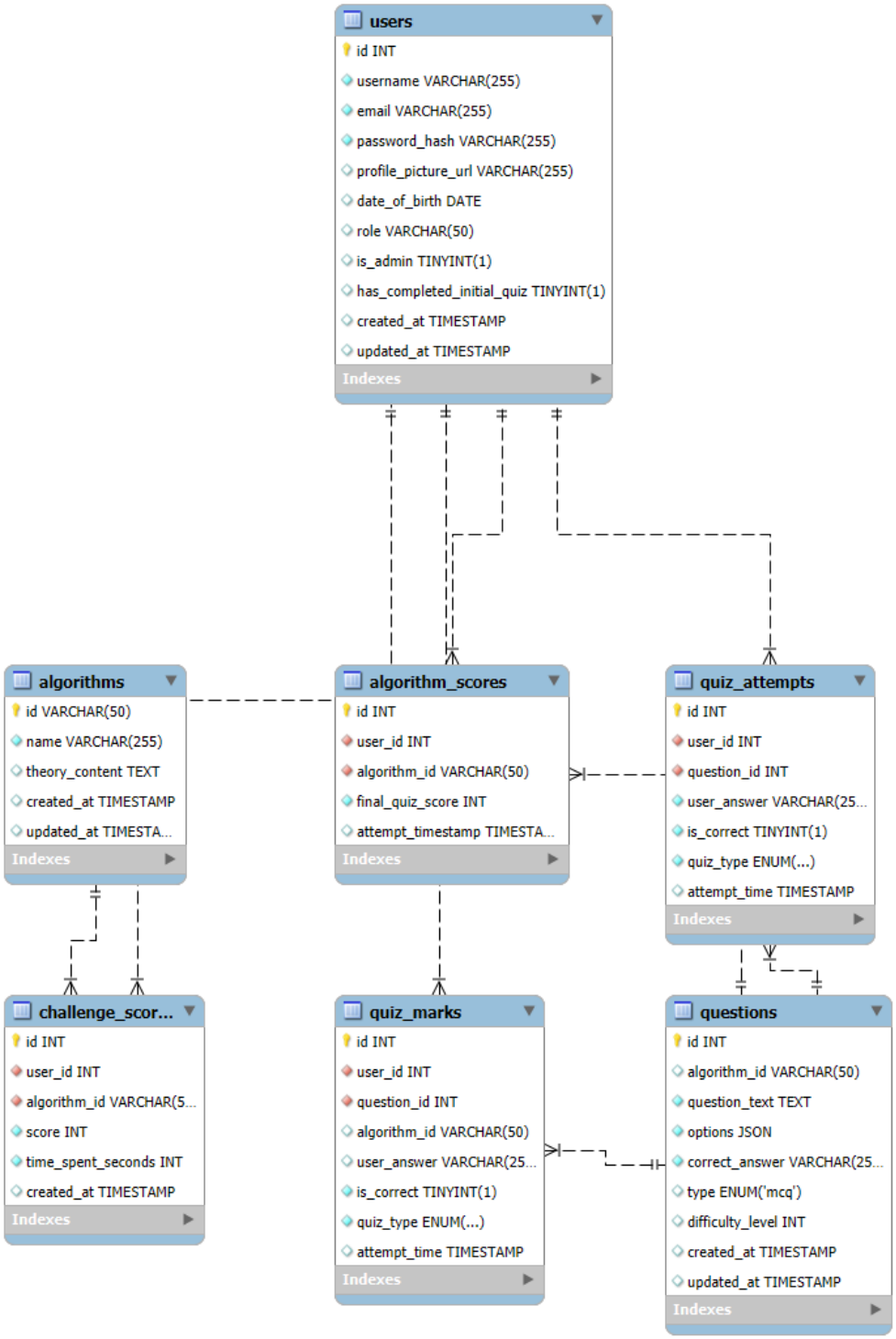


Figure 4.2: ER Diagram of sorting-arcade-db

The ER diagram illustrates relationships such as:

- One-to-Many between users and scores (a user can attempt multiple challenges).
- One-to-Many between questions and scores (scores depend on specific question sets attempted).

This relational structure was chosen because of its simplicity, integrity enforcement, and compatibility with backend APIs.

4.4 User Interface Design

- The interface was carefully designed for usability, accessibility, and consistency. The major pages include:
- Homepage: Introduces sorting algorithms, explaining their importance in computer science. The page contains a “Get Start” button that navigates to the algorithm selection page.
- Registration/Login Page: Facilitates authentication. If a user attempts to start a challenge without being logged in, the system redirects them to this page.
- Algorithm Selection Page: Lists five sorting algorithms (Bubble, Insertion, Selection, Merge, Quick). Users select one to begin their learning journey.
- Challenge Pages: Each algorithm has a dedicated challenge page tailored to its sorting mechanism. Visual bars, pivot elements, or split/merge boxes are displayed according to the chosen algorithm.
- MCQ Pages: Before and after gameplay, users answer MCQs.
- Leaderboard and Profile Page: Shows global ranking of users as well as personal performance details.

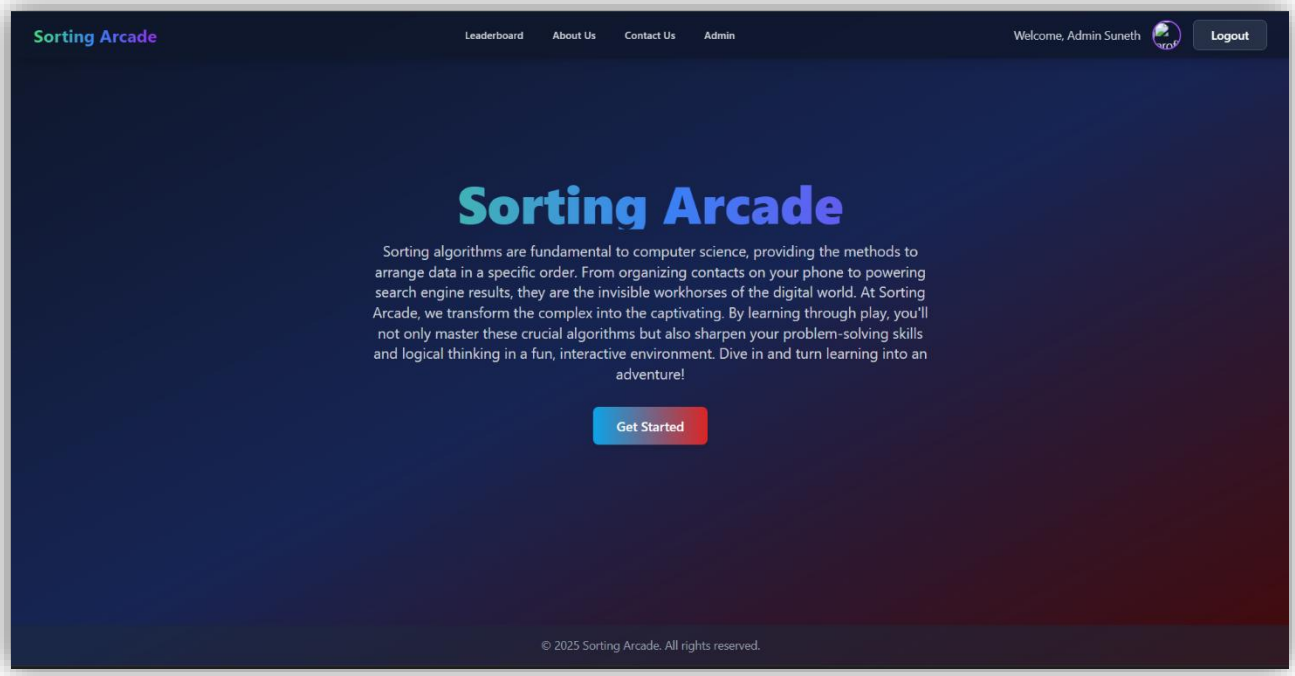


Figure 4.3: Home page

By applying Tailwind CSS, the design maintained responsiveness across desktop and mobile devices. The consistent use of colors, button layouts, and visual bars enhances the learning experience by reducing cognitive load.

4.5 Functional Module Implementation

4.5.1 Bubble Sort Module

The Bubble Sort module consists of six levels, each progressively more complex. Users are presented with an array represented by vertical bars. Two buttons Swap and No Swap are available. At each step, the user must decide whether to swap adjacent bars or leave them in place. A wrong decision results in immediate failure and restarting from current level.

The implementation leverages React's state management to dynamically update array states, while the backend logs each attempt. Scoring is time-based: completing within 20 seconds awards 100 marks, with deductions of 5 marks for every additional 5 seconds.

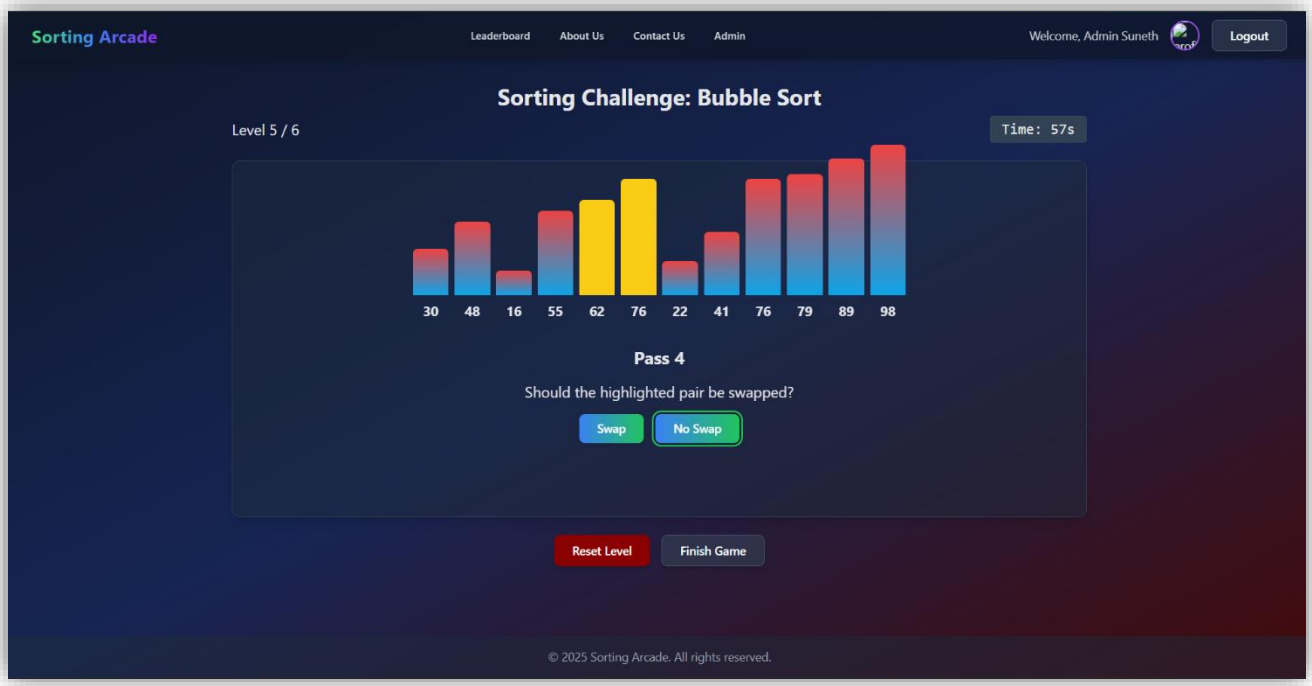


Figure 4.4: Bubble sort game play

4.5.2 Insertion Sort Module

In this module, the system simulates step-by-step insertion of elements. Users must choose the correct position for the current element. For example, given [55, 87, 45], the user is prompted to insert 45 into the correct place among already sorted elements.

If the user selects the wrong position, the challenge fails, and they must restart the level. This module was implemented using React drag-and-drop components, ensuring an intuitive learning experience.

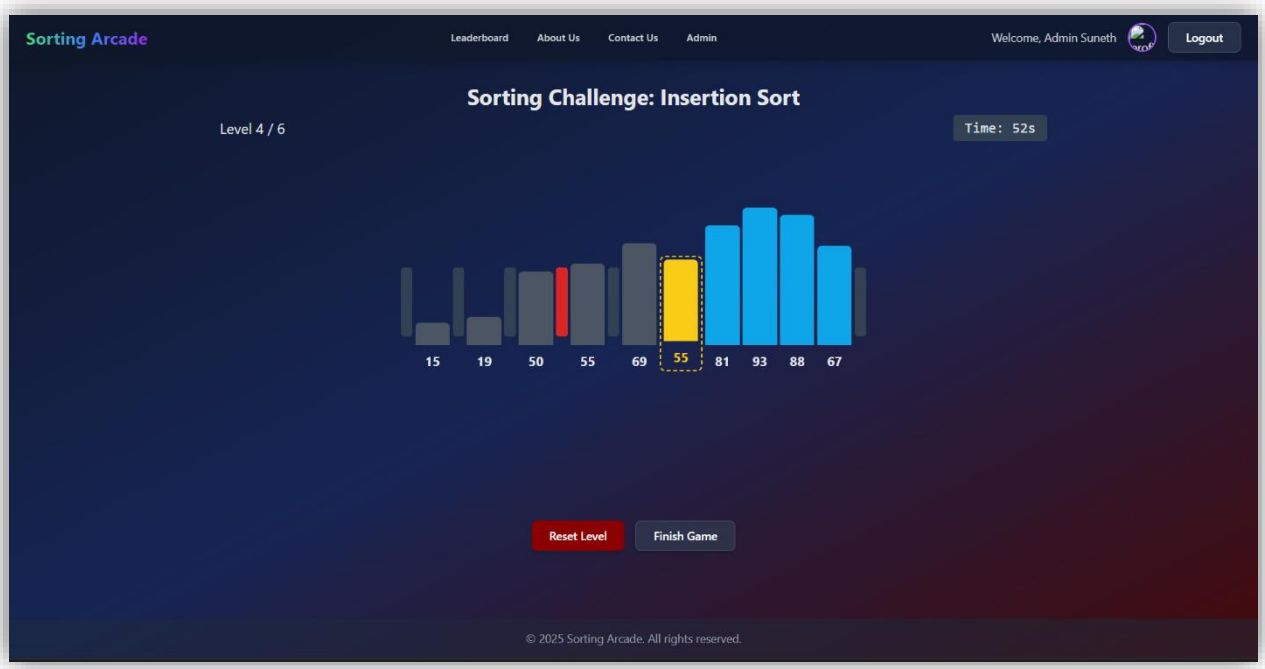


Figure 4.5: Insertion sort game play

4.5.3 Selection Sort Module

The Selection Sort challenge displays vertical bars in gradient colors (red and blue). The user must repeatedly select the smallest element. Correct selections cause bars to turn gray, visually indicating sorted status.

If an incorrect bar is selected, the level fails. Implementation uses event listeners to track user clicks and updates states accordingly. The backend records whether the selection was correct or incorrect.

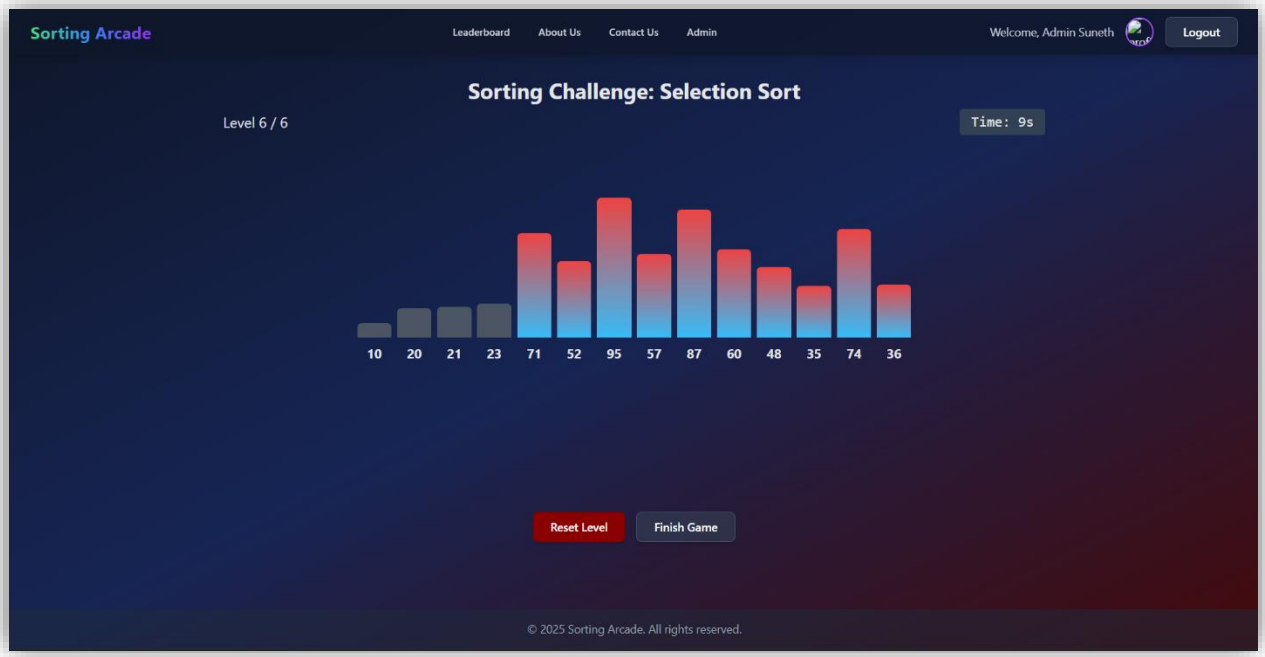


Figure 4.6: selection sort game play

4.5.4 Merge Sort Module

This module simulates the recursive breakdown and merging process of Merge Sort. Users have two options: Split and Merge. Initially, they split arrays into halves until reaching single elements. Then, they perform merging by typing the sorted sequence of selected subsets.

Frontend implementation employs React forms to capture input, while validation logic ensures correctness of splits and merges. Errors force the user to restart the level.

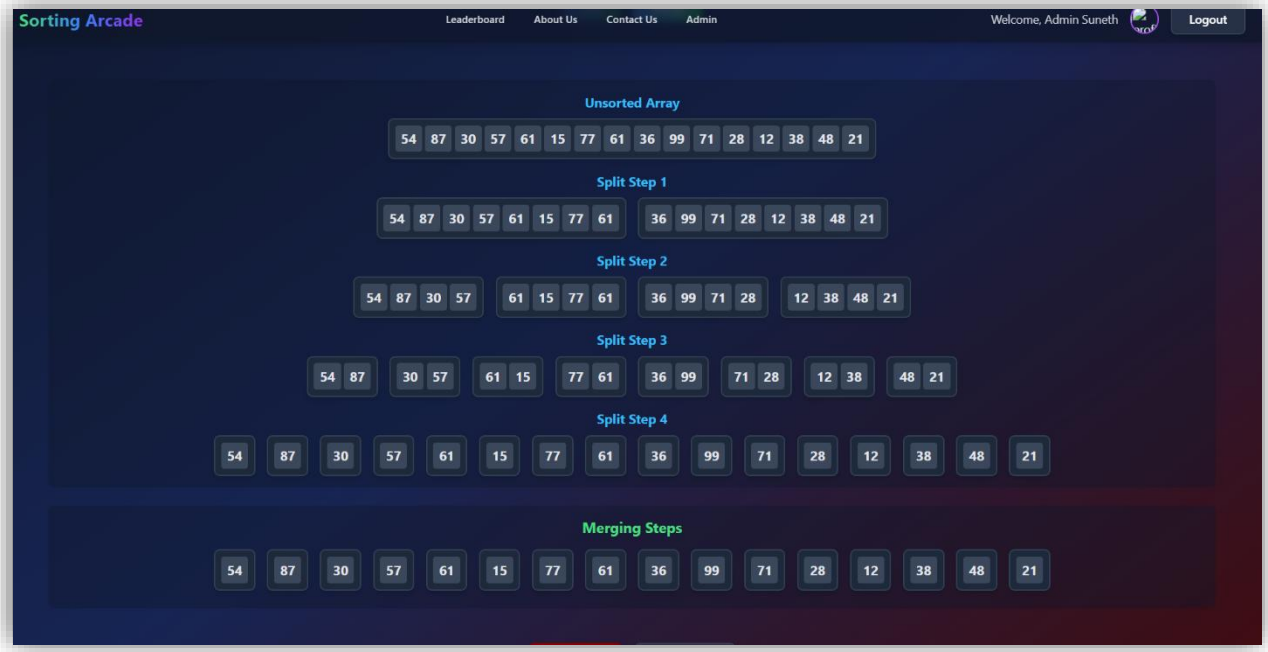


Figure 4.7: Merge sort game play

4.5.5 Quick Sort Module

The Quick Sort module emphasizes partitioning using a pivot element, highlighted in yellow. Two boxes are displayed: Less than Pivot and Greater than Pivot. Users drag and drop elements accordingly. The system iterates this process until the array is completely sorted.

React drag-and-drop libraries were used for implementation. Backend validation ensures that each partition is correct before proceeding to the next pivot.

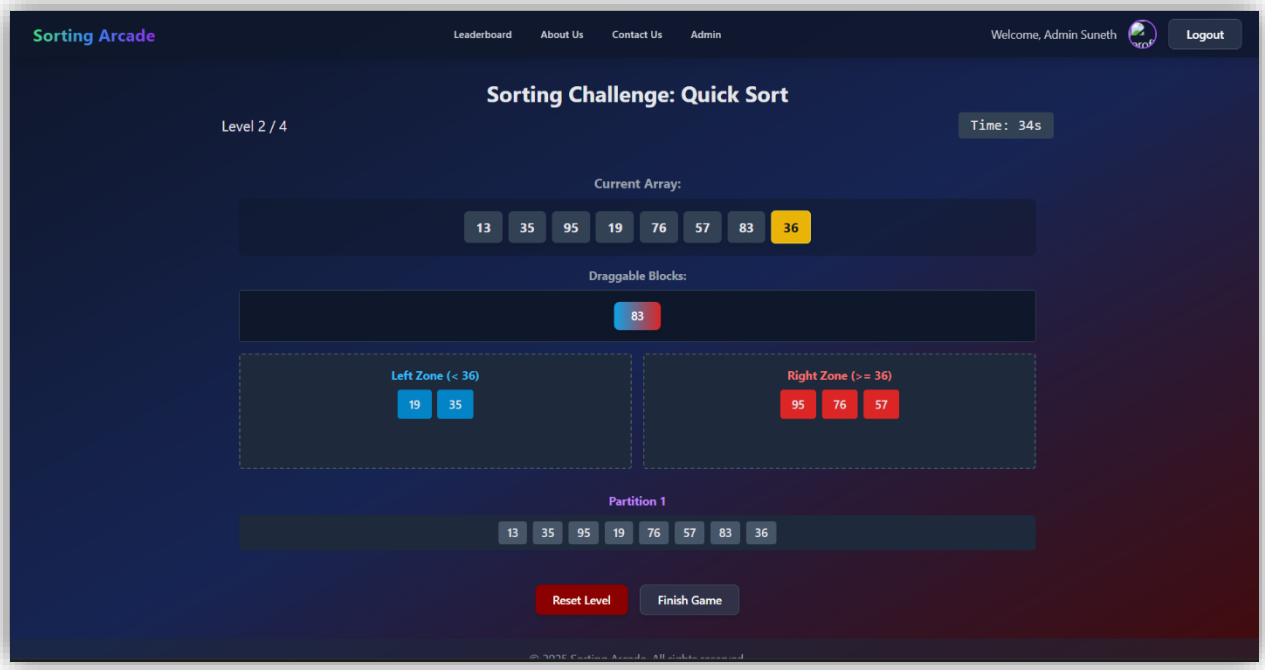


Figure 4.8:Quick sort game play

4.6 Assessment System

Assessment is integral to the system.

- Initial MCQs: 5 randomly generated questions from the questions table test a user's baseline knowledge before starting a challenge.
- Final MCQs: 10 algorithm-specific questions assess knowledge gained.
- Scoring Mechanism: Total score is calculated from MCQ performance and challenge completion time. Scores are logged in the scores table and aggregated in the leaderboard.

The randomization of MCQs was implemented using SQL RAND() functions, ensuring fairness and variety for each attempt.

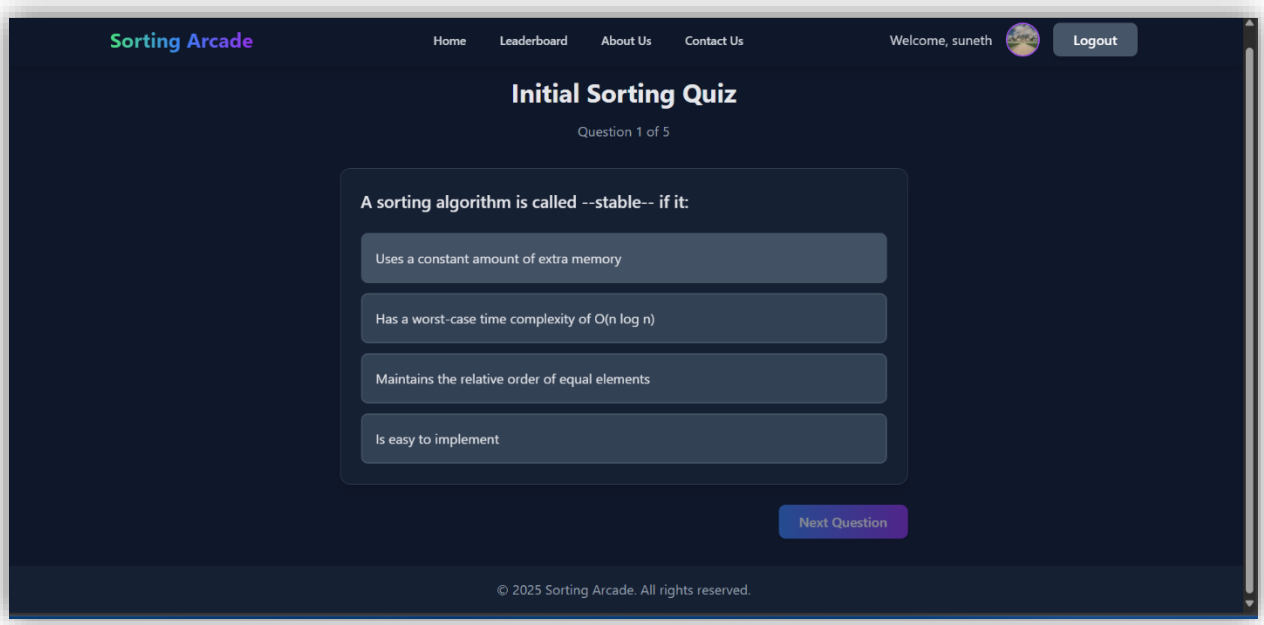


Figure 4.9:Initial MCQ section

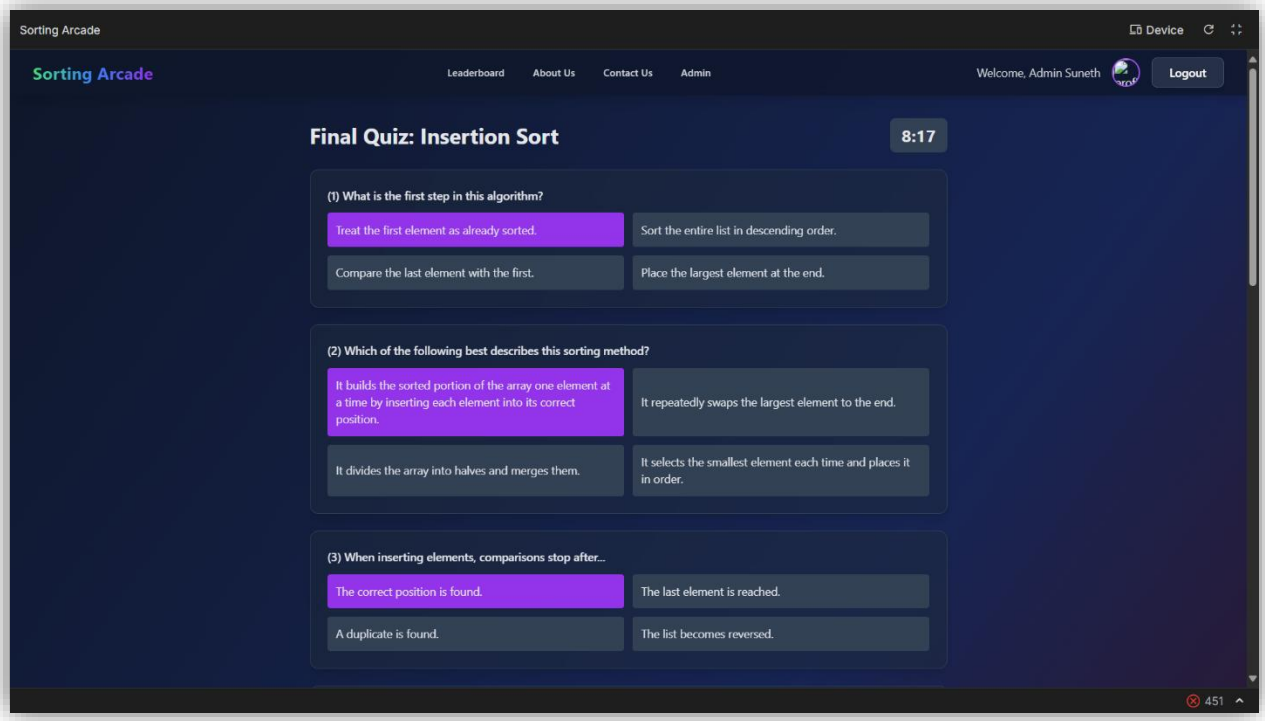


Figure 4.10:final MCQ section- Insertion sort

4.7 Leaderboard and Administration Features

The leaderboard enhances competitiveness and engagement by displaying scores of all users ranked globally. Each user can also access their personal profile to view detailed performance metrics.

Administrators have special access to dashboards that display:

- Initial MCQ scores,
- Challenge performance,
- Final MCQ scores.

This feature allows instructors to analyze learning progress at both individual and group levels.

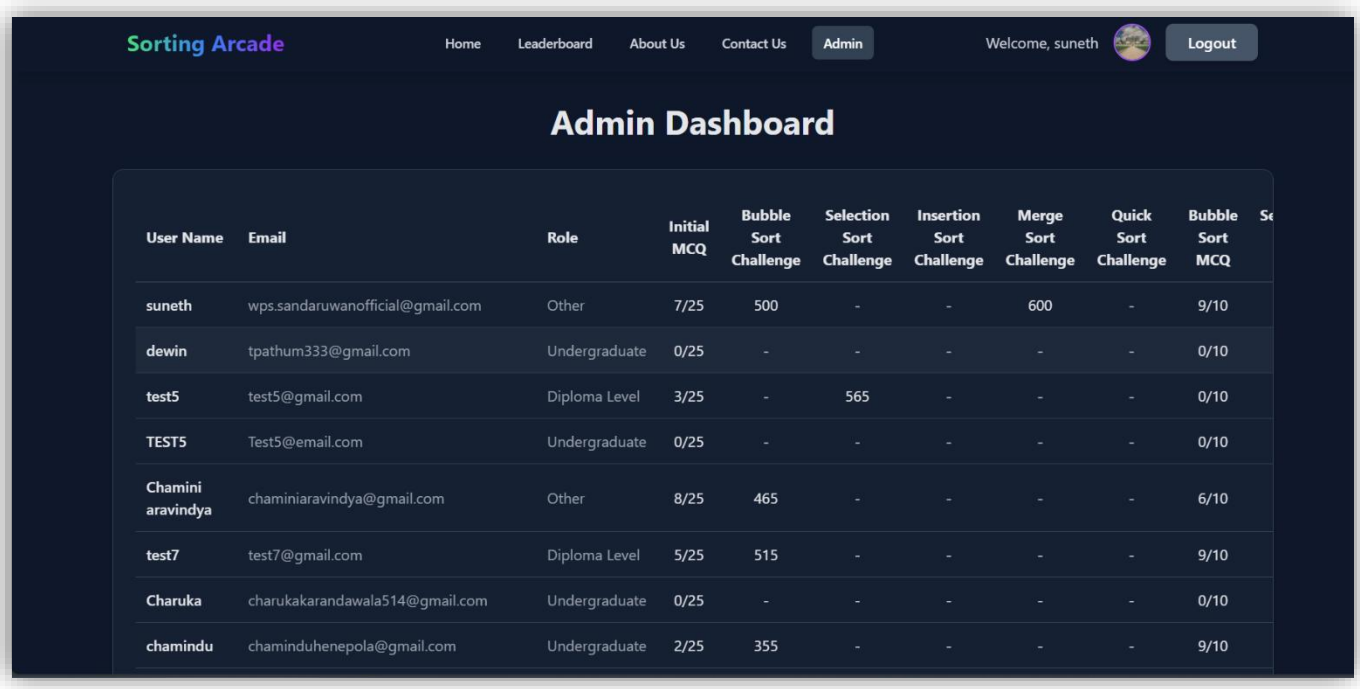


Figure 4.11:Admin dashboard

4.8 Deployment

The deployment strategy ensured accessibility and scalability:

- Frontend: Deployed on Netlify.
- Backend: Hosted on Google Cloud Console using Node.js environment.
- Database: Cloud-based MySQL instance integrated with backend APIs.
- System URL: sortingarcade.netlify.app

The deployment pipeline was designed for reliability, allowing seamless updates and minimal downtime.

4.9 Summary

This chapter detailed the design and implementation of Sorting Arcade, covering architecture, database structure, user interface, core modules, assessment design, leaderboard, and deployment strategy. The integration of interactive gameplay with theoretical learning provides an innovative platform for teaching sorting algorithms. With a robust architecture and scalable design, the system achieves its intended purpose of engaging students in active and meaningful learning.

Chapter 5 – Implementation and Design

5.1 Introduction

This chapter presents the implementation process and design considerations of the Sorting Arcade system. Building on the system requirements and architecture described in Chapter 4, this chapter provides a detailed account of the development workflow, algorithm visualization mechanisms, data handling strategies, and the overall integration of educational and gaming elements. In addition, the chapter describes the logic behind individual modules, the pseudocode used in their implementation, and justifies the selection of programming languages, frameworks, and tools adopted for the project. The chapter concludes with key implementation outcomes and highlights significant contributions unique to this research.

5.2 Development Process

The development of Sorting Arcade followed an iterative and incremental approach aligned with agile principles. Each iteration focused on implementing a functional module (e.g., Bubble Sort challenge, leaderboard system, MCQs), followed by testing and refinement based on feedback. The modular development approach enabled parallel progress across the frontend, backend, and database.

The major stages included:

1. Frontend Implementation (React.js + Tailwind CSS): Designing user-friendly interfaces, visualization of sorting algorithms, and interactive gameplay logic.

2. Backend Implementation (Node.js + Express.js): Building RESTful APIs to handle user authentication, question retrieval, score submission, and leaderboard updates.

3. Database Integration (MySQL): Structuring tables for users, scores, and questions; ensuring secure communication with the backend.

4. Deployment (Netlify + Google Cloud Console): Configuring scalable cloud-based hosting solutions for the application.

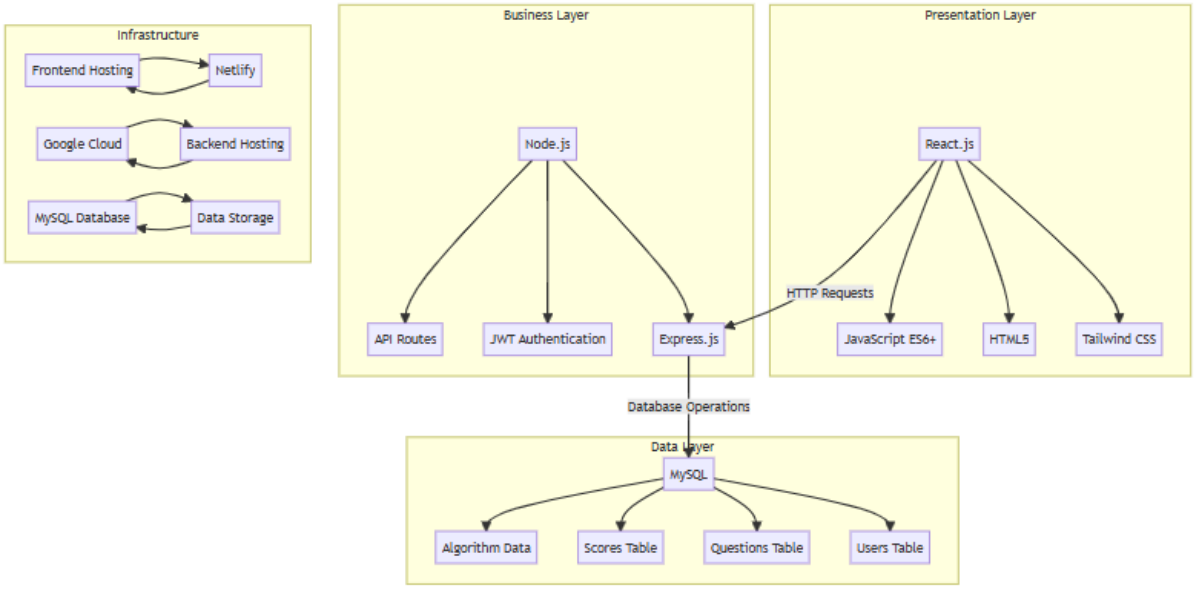


Figure 5.1:Technology stack

5.3 Algorithmic Logic and Pseudocode

The system implements five sorting algorithms (Bubble, Insertion, Selection, Merge, Quick) through gamified challenges. Each challenge transforms algorithmic logic into interactive steps.

5.3.1 Bubble Sort Implementation

Algorithmic Logic:

1. Display a random unsorted array of vertical bars.
2. Users compare adjacent bars sequentially:
    - If the first bar > second bar → click Swap.
    - Else → click No Swap.
3. If the user makes an incorrect decision, the level restarts level .
4. Each level increases the number of elements or complexity.
5. Completion time determines the score:
    - ≤20 seconds → 100 marks.
    - Each additional 5 seconds → reduce 5 marks.

Pseudocode:

buble.txt

repos > pusdo > buble.txt

```
for each level in bubble_sort_challenge:
    array = generate_random_array(level)
    while array not sorted:
        for i = 0 to length(array)-2:
            if user clicks 'Swap' and array[i] > array[i+1]:
                swap(array[i], array[i+1])
            else if user clicks 'No Swap' and array[i] <= array[i+1]:
                continue
            else:
                restart_level()
calculate_score(time_taken)
```

Figure 5.2:Bubble sort pusdo code

5.3.2 Insertion Sort Challenge

Algorithmic Logic:

1. Display an unsorted array of vertical bars.
2. Assume the first element is sorted.
3. User selects the correct position for each subsequent element among sorted elements.
4. Wrong selection → restart level.
5. Repeat until all levels are completed.

Pseudocode:

insertion.txt

repos > pusdo > insertion.txt

```
for each level in insertion_sort_challenge:
    array = generate_random_array(level)
    sorted_array = [array[0]]
    for i = 1 to length(array)-1:
        user_selected_position = get_user_input(array[i])
        if correct_position(user_selected_position, sorted_array):
            insert(array[i], sorted_array, user_selected_position)
        else:
            restart_level()
calculate_score(time_taken)
```

Figure 5.3:Insertion sort pusdo code

5.3.3 Selection Sort Challenge

Algorithmic Logic:

1. Display an unsorted array of colored vertical bars.
2. User identifies the smallest remaining element and moves it to the correct position.
3. Correct selection → bar color changes to gray, indicating sorted status.
4. Incorrect selection → restart level.
5. Repeat until all elements are sorted.

Pseudocode:

selection.txt

repos > pusdo > selection.txt

```
for each level in selection_sort_challenge:
    array = generate_random_array(level)
    for i = 0 to length(array)-1:
        smallest_index = find_smallest(array[i:end])
        if user_clicks_correct(smallest_index):
            mark_as_sorted(array, smallest_index)
        else:
            restart_level()
calculate_score(time_taken)
```

Figure 5.4:selection sort pusdo code

5.3.4 Merge Sort Challenge

Algorithmic Logic:

1. Display an unsorted array.
2. User clicks Split → array divides into two subarrays.
3. User enters elements for left and right halves until arrays contain single elements.
4. User clicks Merge → merges two subarrays in sorted order.
5. Repeat merging until the final sorted array is formed.
6. Incorrect entry → restart level.

Pseudocode:

merge.txt

repos > pusdo > merge.txt

```
for each level in merge_sort_challenge:
    array = generate_random_array(level)
    while array not fully sorted:
        subarrays = split_array(array)
        for each subarray:
            user_input = get_user_split_input(subarray)
            if user_input != correct_split(subarray):
                restart_level()
        merged_array = merge_subarrays(subarrays)
calculate_score(time_taken)
```

Figure 5.5:merge sort pusdo code

5.3.5 Quick Sort Challenge

Algorithmic Logic:

1. Display an unsorted array. Highlight the pivot element in yellow.
2. User drags and drops elements into less-than pivot or greater-than pivot boxes.
3. Correct placement → system proceeds to next pivot iteration.
4. Incorrect placement → restart level.
5. Repeat until array is fully sorted.

Pseudocode:

quick.txt

repos > pusdo > quick.txt

```
for each level in quick_sort_challenge:
    array = generate_random_array(level)
    pivot = select_pivot(array)
    while array not sorted:
        for each element in array excluding pivot:
            user_box = get_user_drop(element)
            if correct_box(element, pivot, user_box):
                continue
            else:
                restart_level()
        pivot = next_pivot(array)
calculate_score(time_taken)
```

Figure 5.6:quick sort pusdo code

5.4 Design Workflow

The system's workflow was designed to integrate theory, practice, and assessment seamlessly.

1. Theory Page: Each algorithm is accompanied by a brief description and relevance.
2. Initial MCQs: A pre-assessment stage measuring baseline knowledge.
3. Interactive Challenge: Users complete algorithm-specific challenges with increasing difficulty.
4. Final MCQs: A post-assessment stage evaluating learning gains.
5. Scoring & Leaderboard: Performance is logged, and rankings are updated in real-time.

5.5 Technology Justification

The choice of technologies was carefully made to balance performance, scalability, and usability

Component	Technology	Justification
Frontend	React.js, Tailwind	Enables modular UI development, responsive design, and interactive gameplay.
Backend	Node.js, Express.js	Lightweight, scalable, and effective for REST API development.
Database	MySQL	Reliable relational database with strong support for structured queries.
Hosting	Netlify (Frontend), Google Cloud Console (Backend)	Provides scalability and CI/CD integration.
Libraries	React DnD, React Router	Facilitates drag-and-drop sorting and routings

Table 5.1: technology justification

5.6 Unique Contributions

Unlike traditional algorithm visualizations, Sorting Arcade integrates active gameplay with formative assessments (MCQs), thereby enhancing engagement and knowledge retention. Features such as dynamic scoring, immediate failure feedback, and leaderboards foster both individual learning and competitive motivation.

5.7 Summary

This chapter provided a detailed description of the implementation and design of the Sorting Arcade system. The chapter explained the pseudocode and logic behind each algorithm module, described the user interaction design, and justified the choice of technologies. By blending interactive algorithm visualizations with structured assessments, Sorting Arcade contributes to the field of game-based learning in a novel way. The next chapter discusses testing and evaluation to validate the system's effectiveness and performance.

Chapter 6 – Testing and Evaluation

6.1 Introduction

Testing and evaluation are critical phases of the system development life cycle, ensuring that the Sorting Arcade platform not only functions correctly but also fulfill its primary objective of enhancing student learning through game-based interaction. This chapter discusses the testing methodologies applied to verify system functionality, usability, and performance. Furthermore, it presents the evaluation process used to measure the effectiveness of the system as an educational tool for teaching sorting algorithms. Both technical testing (to confirm correctness, stability, and reliability) and educational evaluation (to measure learning outcomes and engagement) are addressed in this chapter.

6.2 Testing Approach

The testing process combined software testing techniques and educational assessment methods.

1. Software Testing Perspective:
    - Conducted unit testing for individual components (e.g., MCQ module, leaderboard updates).
    - Performed integration testing to verify communication between frontend, backend, and database.
    - Applied system testing to evaluate end-to-end workflow across multiple user sessions.
    - Executed usability testing to ensure intuitive interaction for students.
2. Educational Evaluation Perspective:
    - Designed pre-test and post-test assessments using MCQs to measure conceptual learning.
    - Collected user feedback on usability, engagement, and motivation.
    - Compared performance data (scores, completion times, and accuracy rates) with traditional learning methods.

6.3 Functional Testing

Functional testing validated whether the system modules operated according to requirements outlined in Chapter 3. Each feature was tested against expected outcomes.

- User Registration and Authentication: Verified account creation, login, and password recovery mechanisms using both valid and invalid credentials.
- MCQ Module: Tested randomization of questions, correctness validation, and score calculation.
- Algorithm Challenges: Ensured Bubble, Insertion, Selection, Merge, and Quick Sort challenges followed algorithmic rules and detected incorrect moves.
- Leaderboard: Checked for real-time updates, correct ranking, and tie-handling mechanisms.
- Data Persistence: Confirmed user data, scores, and game history were securely stored in the database.

Feature / Module	Description	Test Case / Action	Expected Result	Actual Result	Status (Pass/Fail)
User Registration	Register new users	Enter valid name, email, password	User account created	User account created	Pass
Login	Login with registered credentials	Enter email and password	Access to dashboard	Access granted	Pass
Login	Login with unregistered credentials	Enter email and password	Display the error massage	Display the error massage	Pass
Bubble Sort Challenge	Swap/No Swap functionality	Play level 1 challenge	Correct behavior for swap/no swap	Incorrect behavior(swapped correctly ordered element)	Fail
Bubble Sort Challenge	Swap/No Swap functionality	Play level 1 challenge	Correct behavior for swap/no swap	Behaves as expected	Pass
Insertion Sort Challenge	Correct insertion of element	Play level 2 challenge	Element inserted at correct position	Works correctly	Pass
Selection Sort Challenge	Pick smallest element	Play level 3 challenge	Sorted elements marked	Works correctly	Pass

			correctly		
Merge Sort Challenge	Split & Merge functionality	Split array and merge subarrays	Merged array correct	Works correctly	Pass
Quick Sort Challenge	Drag & drop elements into pivot boxes	Arrange elements according to pivot	Sorted array achieved	Works correctly	Pass
Pre-Test MCQs	Answer questions	Select answers for 5 MCQs	Correctly recorded	Recorded	Pass
Post-Test MCQs	Answer questions	Select answers for 10 MCQs	Correctly recorded	Recorded	Pass
Leaderboard	View scores	Open leaderboard	Shows top scores	Shows correctly	Pass

Table 6.1:test cases table

6.4 Usability Testing

Usability was a primary concern since the platform targeted students who may not have prior programming experience. Testing was conducted with a group of 20 undergraduate students from NSBM Green University.

The following usability aspects were evaluated:

1. Ease of Navigation: Students reported that the transition between , theory → MCQs → challenges was smooth.
2. Clarity of Instructions: Tutorials and on-screen prompts were rated as "clear" by 85% of participants.
3. Engagement Level: The gamified nature (scoring, competition, and immediate feedback) increased motivation compared to static visualization tools.
4. Error Recovery: Participants could easily restart challenges without losing motivation.

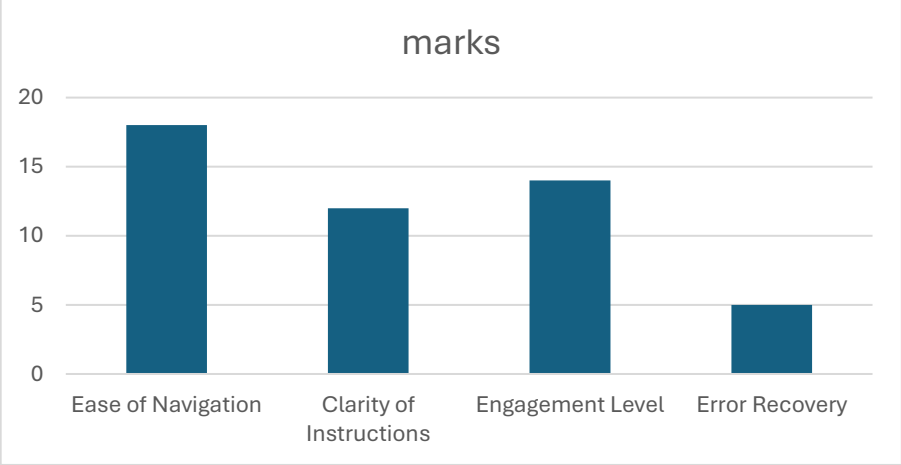


Figure 6.1: bar chart of usability survey

6.5 Educational Effectiveness Evaluation

The primary objective of this research was to evaluate the educational effectiveness of the Sorting Arcade platform in facilitating the learning of sorting algorithms. Since the system was designed to integrate theoretical learning with interactive challenges, the evaluation focused on measuring students' knowledge improvement by comparing their performance in the pre-assessment (initial MCQs) with the post-assessment (final MCQs).

Participants and Data Collection

A total of 20 undergraduate students participated in the evaluation. Each student was required to register on the Sorting Arcade platform and attempt at least one sorting algorithm challenge. For each chosen algorithm, the system automatically administered two sets of assessments:

- Initial MCQ Test (Pre-Test): Five randomly generated multiple-choice questions (MCQs) from the database to measure prior knowledge.
- Final MCQ Test (Post-Test): Ten algorithm specific MCQs presented after completing all challenge levels, designed to measure knowledge gained.

The platform recorded scores for both pre-test and post-test attempts, which were stored in the database for analysis.(initial and final mcqs are provided in Appendix A)

Methodology

The evaluation followed a pre-test/post-test design. No separate control group was employed. Instead, the focus was on measuring learning gains within the same group of participants after exposure to the game-based learning environment.

The average pre-test and post-test scores across all participants were compared to determine the improvement attributable to the platform. The improvement was expressed both in absolute score increase and percentage gain.

Results

The results revealed a substantial improvement in students' understanding of sorting algorithms after engaging with the Sorting Arcade:

- Group Initial Average : 64.40%
- Group Final Average : 80.70%
- Overall Group Improvement : 16.30%

The analysis was conducted on data collected from 20 students. The group initial average score was 64.40%, while the final average score increased to 80.70%, resulting in an overall improvement of 16.30%. (See figure 6.2 pre vs post MCQ answer average)A detailed breakdown of individual student scores (initial MCQs and final MCQ marks) is provided in Appendix B

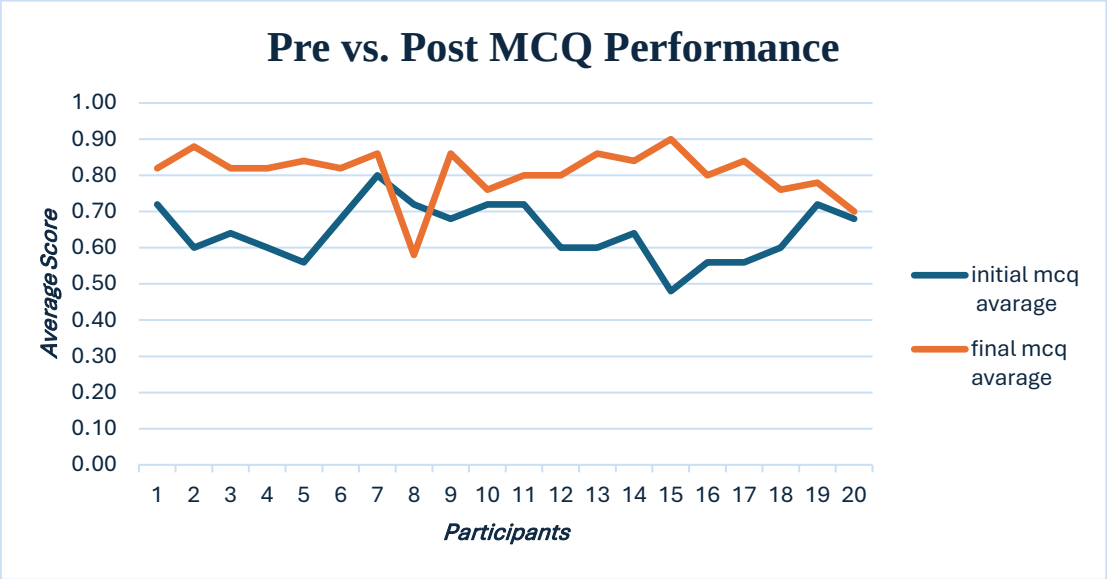


Figure 6.2 pre vs. post MCQ answer average

6.6 Discussion

The findings of this study validate the educational effectiveness of the Sorting Arcade platform, demonstrating that the integration of game-based mechanics, algorithm visualization, and interactive problem-solving activities can significantly enhance learners' engagement, motivation, and knowledge retention. These results are consistent with prior studies, which have emphasized that sorting algorithms are often perceived as abstract and challenging by students, and that traditional teaching methods fail to adequately support conceptual understanding [1], [2]. By embedding learning into an interactive and gamified environment, Sorting Arcade directly addresses these challenges and provides learners with an engaging, student-centered alternative to conventional pedagogy.

The improvements (16.30%) in pre- and post-assessment scores highlight that learners not only developed factual recall of algorithmic processes but also demonstrated stronger procedural knowledge. This aligns with Abdul Rauf et al. [1], who reported that conventional lectures often lead to superficial memorization rather than genuine problem-solving skills. Similarly, Alqahtani [2] noted that passive learning approaches rarely cater to diverse learning styles, leading to disengagement and low retention. By contrast, the multi-level challenges and immediate feedback embedded in Sorting Arcade created an active learning environment, reinforcing understanding at each stage of algorithm execution.

The use of visualization also proved critical, echoing findings from the Visualgo project [3], where step-by-step algorithm animations enhanced students' comprehension of abstract concepts. However, unlike static visualizations, Sorting Arcade required learners to actively interact with arrays, pivots, and sorting steps, thereby promoting deeper cognitive engagement. This supports the growing body of research suggesting that interactive systems outperform purely demonstrative tools in developing algorithmic thinking skills [4].

In terms of gamification, the study's results mirror those of Nok and Chusri [4], who found that game-based learning significantly improved both motivation and satisfaction in teaching sorting algorithms. Similarly, Fürlinger [5] stressed the importance of usability and intuitive design in ensuring effective learning outcomes from educational games. The positive reception and improved performance observed in this study suggest that Sorting Arcade successfully balanced both game mechanics and pedagogical usability.

Despite these promising results, some limitations must be acknowledged. The relatively small sample size and single-institution setting may reduce generalizability, and the lack of a control group using lecture-based methods prevents definitive causal claims. Future studies could adopt a controlled experimental design, comparing groups exposed to gamified versus traditional instruction, and applying statistical tests to validate differences. Additionally, longitudinal studies are necessary to examine whether skills acquired via gamified platforms persist over time and transfer to broader problem-solving contexts.

In summary, this discussion reinforces the insights from the literature review: traditional teaching methods often fail to address student difficulties in learning sorting algorithms, while interactive visualization [3], gamification [4], and usability-focused design [5] provide powerful tools to bridge this gap. The Sorting Arcade contributes to this growing research area by combining these approaches into a cohesive learning platform, demonstrating measurable improvements in algorithm comprehension and engagement.

Study / Approach	Reported Outcome	Sorting Arcade Outcome (This Study)
Abdul Rauf et al. (2020)	Students struggle with abstract algorithm concepts.	Improved conceptual understanding via interactive gameplay.
Alqahtani (2018)	Passive teaching reduces engagement & retention.	Higher engagement and retention through gamified challenges.
Visualgo (n.d.)	Animations improve comprehension.	Interactivity + visualization deepened comprehension.
Nok & Chusri (2019)	Game-based learning improves performance & motivation.	Significant post-test improvement and positive learner feedback.
Fürlinger (2024)	Usability is key to effective educational games.	User-friendly design ensured accessibility and learning effectiveness.

Table 6.3:reported vs this study comparison

6.7 Summary

This chapter presented the testing and evaluation of the Sorting Arcade system. Functional testing verified that all system components worked as intended, while usability testing ensured an intuitive user experience. and educational evaluation demonstrated the significant learning gains achieved through the gamified approach. These results validate the feasibility of the system as both a teaching aid and a research contribution. The next chapter will conclude the research by summarizing findings and suggesting future work.

Chapter 7 – Concluding Remarks

7.1 Accomplishment of the Research Objectives

The primary objective of this research was to design, implement, and evaluate a game-based learning platform "Sorting Arcade" to address the persistent challenges students face when learning sorting algorithms. The study successfully demonstrated that gamification, when combined with interactive visualization and usability-driven design, can significantly enhance learners' comprehension and engagement.

Through pre- and post-assessments, learners showed notable improvement in understanding the selected sorting algorithms (Bubble, Insertion, Selection, Merge, and Quick Sort). The system also incorporated multiple-choice questions (MCQs) before and after gameplay, which triangulated learning outcomes. These findings align with Abdul Rauf et al. [1], who identified the difficulty of algorithm learning in traditional classrooms, and with Nok and Chusri [5], who showed that gamified approaches increase motivation and comprehension.

7.2 Problems Encountered

During the development and evaluation of the Sorting Arcade platform, several significant challenges were encountered that impacted both the implementation and testing phases.

1. Limited Expertise in Frontend and Backend Development:

The development of the application required extensive knowledge of React.js for the frontend and Node.js with Express.js for the backend. Initially, the limited prior experience with these technologies presented challenges in designing a scalable, responsive, and interactive user interface. Implementing features such as the authentication system, user session management, and secure registration/login mechanisms required extensive learning and debugging, which increased development time.

2. Complexity of Algorithm Gamification:

Translating abstract sorting algorithms, particularly Merge Sort and Quick Sort, into interactive game-based mechanics was inherently complex. Merge Sort required splitting arrays into multiple subarrays and merging them interactively, which led to numerous logic errors and synchronization issues. Quick Sort posed additional challenges with pivot selection, drag-and-drop functionality, and real-time iteration updates, which required careful design and multiple iterations to ensure correctness.

3. Game Logic and Real-Time Validation:

Implementing real-time feedback for users' actions was challenging, as it required accurately checking user inputs against the expected algorithmic behavior. For example, in Bubble Sort, the system needed to validate whether a "swap" or "no swap" action was correct, immediately updating the game state and handling incorrect attempts. Designing these mechanisms while maintaining smooth performance and preventing lag was a critical obstacle.

4. Integration with Cloud-Based Database and Hosting:

Hosting the backend on Google Cloud Console and integrating it with the frontend hosted on Netlify required careful handling of API endpoints, CORS policies, and database connections to MySQL.

5. User Experience and Usability Considerations:

Designing a user interface that was intuitive, visually appealing, and pedagogically effective required balancing gameplay complexity with usability. Ensuring that users could easily understand interactive elements such as vertical bars, draggable items, split/merge boxes, and pivot highlights necessitated multiple UI redesigns and usability testing iterations.

6. Testing and Evaluation Limitations:

Evaluating the educational effectiveness of the platform was constrained by a limited sample size and the absence of a traditional control group. This restricted the ability to draw strong comparative conclusions about the efficacy of gamified learning over traditional methods, highlighting the need for more extensive future studies.

7. Time Management and Scope Limitations:

Due to the ambitious nature of the project, managing the timeline while maintaining quality across multiple game modules, levels, and MCQ integrations was a persistent challenge.

Despite these challenges, the project successfully delivered a functional, user-centered, and gamified learning platform, and the difficulties encountered provided valuable insights for both technical development and educational design.

7.3 Self-Reflection

7.3.1 Research Ideology

The development of the Sorting Arcade platform reinforced the resercher's belief in pragmatic, learner-centered approaches to computer science education. By combining interactive visualizations, gamification, and usability-focused design, the study aimed to bridge the gap between theoretical knowledge and practical understanding. The research process demonstrated that integrating technology and pedagogy requires not only technical proficiency but also thoughtful design decisions grounded in educational principles [1], [2].

7.3.2 Skills and Knowledge Acquired

- The project significantly enhanced the researcher's technical and educational skills:
- Frontend and Backend Development: Extensive hands-on experience with React.js, Tailwind CSS, Node.js, Express.js, and MySQL improved the researcher's ability to develop scalable web applications with interactive features and secure authentication mechanisms.
- Algorithm Implementation in Interactive Environments: Translating sorting algorithms, especially Merge Sort and Quick Sort, into engaging gameplay enhanced understanding of algorithmic logic, data structures, and real-time validation mechanisms.
- Usability and User Experience Design: Iterative design and testing improved the researcher's competence in creating intuitive interfaces that support diverse learning styles while maintaining engagement.
- Cloud Deployment and Integration: Hosting the backend on Google Cloud Console and frontend on Netlify provided practical experience in deploying and integrating cloud-based applications, handling API requests, and maintaining data consistency.

7.3.3 Reflection on Challenges

Encountering challenges during development was a critical aspect of the learning process. Limited prior experience with advanced frontend and backend frameworks required extensive self-learning, debugging, and problem-solving, which strengthened technical resilience. Algorithm gamification highlighted the difficulty of representing abstract processes in interactive formats, reinforcing the importance of meticulous planning and testing. Furthermore, issues with system integration and real-time validation emphasized the necessity of robust software engineering practices. These challenges collectively contributed to both personal and professional growth.

7.3.4 Educational Insight and Pedagogical Perspective

The reflection process underscored the value of design-driven research in educational technology. By critically analyzing learner interactions, gameplay outcomes, and user feedback, the researcher gained insight into how gamification can enhance comprehension, motivation, and engagement in algorithmic learning. This perspective aligns with prior literature emphasizing active learning, visualization, and gamification as effective pedagogical strategies.

7.3.5 Personal and Professional Growth

Beyond technical knowledge, the project cultivated broader research competencies:

- Problem-Solving and Critical Thinking: Designing complex game mechanics and addressing unexpected errors in algorithm implementation honed analytical thinking.
- Project Management and Time Management: Managing multiple interconnected modules, deadlines, and evaluation processes improved organizational skills and efficiency.
- Research Communication: Documenting methodology, evaluation, and reflections strengthened academic writing and the ability to convey complex technical information clearly.

In conclusion, the self-reflection demonstrates that the research process itself through technical challenges, iterative design, and pedagogical evaluation was as valuable as the final platform in fostering both personal development and contributions to the domain of computer science education.

7.4 Business Insight of the Idea

The Sorting Arcade platform demonstrates strong potential for practical deployment within educational and commercial settings. In the context of computer science education, the platform can serve as a complementary tool for introductory programming courses, algorithm tutorials, and online learning programs. Its combination of gamification, interactive visualization, and adaptive challenge levels can foster active engagement, motivation, and improved retention among learners [1], [5].

Furthermore, the platform includes features such as leaderboards, user performance tracking, and challenge scores, which promote healthy competition and self-assessment, reinforcing behavioral engagement principles outlined in gamification research [11], [12]. By providing immediate feedback and measurable outcomes, the system enables instructors to identify knowledge gaps and adapt teaching strategies accordingly, making it a valuable learning analytics tool.

From a commercial perspective, Sorting Arcade has potential as an educational technology product (EdTech) that could be licensed to schools, universities, and online course providers. Its

modular design allows for expansion beyond sorting algorithms to other algorithmic concepts, supporting scalable growth. Compared to existing solutions such as Visualgo [3], which primarily focuses on visualization, Sorting Arcade offers a gamified experience that combines assessment, progression, and interactivity, positioning it as a differentiated product in the EdTech market.

7.5 Future Recommendations

While the Sorting Arcade platform effectively addressed the research objectives, several opportunities exist for enhancement and further investigation:

1. Controlled Comparative Evaluation:

Future research should include a controlled study design comparing the gamified platform with traditional lecture-based instruction. This will allow for stronger causal conclusions regarding the impact of gamification on learning outcomes and engagement.

2. Expansion of Algorithmic Scope:

While this study focused on five fundamental sorting algorithms, extending the platform to cover additional algorithmic topics, such as graph traversal, dynamic programming, and searching algorithms, could increase its educational value and applicability to a broader audience.

3. Enhanced Analytics and Reporting:

Developing advanced dashboards for instructors and administrators would provide real-time insights into learner progress, performance trends, and engagement patterns. This could facilitate data-driven instructional interventions and targeted support.

4. User Experience and Accessibility Enhancements:

Future iterations should consider mobile optimization, accessibility compliance (e.g., WCAG standards), and multi-language support to increase inclusivity and global reach.

5. Gamification Elements Expansion:

Additional gamified features such as achievement badges, reward points, and collaborative challenges can further motivate learners and encourage sustained engagement.

6. Longitudinal Studies:

Conducting long-term studies on knowledge retention, learner motivation, and progression will provide stronger evidence for the educational effectiveness of game-based learning in computer science.

8.References

[1] A. Rauf, et al., “Challenges in Learning Sorting Algorithms,” 2020.

[2] A. Alqahtani, “Pedagogical Challenges in Computer Science Education,” 2018.

[3] Visualgo, “Visualgo: Algorithm Visualization Platform,” [Online]. Available: https://visualgo.net

[4] C. Nok and N. Chusri, “Game-based learning for sorting algorithms: The ‘Sorted’ game,” 2019.

[5] T. Fürlinger, “Design and Usability in Game-Based Learning,” 2024.

[6] T. Nguyen, “Limitations of Algorithm Visualization Tools,” 2021.

[7] J. Janani, “Enhance the Understanding of Sorting Algorithms Through Card Ranking Methods (GBL Approach),” IJRISS, 2025.

[8] T. Batsikas and E. Voyiatzaki, “Teaching Sorting Algorithms in 3D Virtual Worlds,” ICERI Proc., 2023.

[9] A. Meta-Study, “Effectiveness of Digital Educational Games in STEM Learning,” Int. J. STEM Educ., vol. 10, no. 36, 2023.

[10] M. R. Al-Khayat, M. U. Gargash, and A. F. Atiq, “Effectiveness of Game-based Learning in Enhancing Students’ Motivation and Cognitive Skills,” JETM, vol. 2, no. 3, pp. 50–62, 2023.

[11] Z. Liu, et al., “Digital Game-based Learning for Programming Education,” Proc. Int. Conf., 2023.

[12] E. Pacheco-Velazquez, V. Rodés, and D. Salinas-Navarro, “Developing Learning Skills through Game-Based Learning in Logistics Education,” JOTSE, vol. 14, no. 1, pp. 169–183, 2024.

[13] A. Study, “Integrating Gamification and Instructional Design for Usability,” Education and Information Technologies, 2022.

[14] B. Research, “Evaluating Educational Game Design Through Adaptive Feedback,” Multimodal Technologies and Interaction, vol. 9, no. 9, 2024.

[15] React Documentation, “React – A JavaScript library for building user interfaces,” [Online]. Available: https://reactjs.org/

[16] Node.js Documentation, “Node.js: JavaScript runtime environment,” [Online]. Available: https://nodejs.org/

[17] Tailwind CSS, “Tailwind CSS Documentation,” [Online]. Available: https://tailwindcss.com/docs

[18] MySQL Documentation, “MySQL 8.0 Reference Manual,” [Online]. Available: https://dev.mysql.com/doc/

[19] Google Cloud, “Hosting Node.js Applications on Google Cloud,” [Online]. Available: https://cloud.google.com/nodejs

[20] Netlify, “Netlify Hosting Documentation,” [Online]. Available: https://docs.netlify.com/

[21] J. K. Lee and H. S. Choi, “The impact of interactive visualization tools on algorithm learning,” *Educ. Tech. Res. Dev.*, vol. 68, pp. 2035–2052, 2020.

Appendices

Appendix A: Initial And Final MCQ samples

id	algorithm_id	question-text	answers	correct_answer
1	Initial	Which of the following is NOT a comparison-based sorting algorithm?	["Merge Sort", "Quick Sort", "Counting Sort", "Bubble Sort"]	Counting Sort
2	Initial	What is the best-case time complexity of a standard Bubble Sort?	["O(n)", "O(n log n)", "O(n^2)", "O(1)"]	O(n)
3	Initial	A sorting algorithm is called "stable" if it:	["Uses a constant amount of extra memory", "Has a worst-case time complexity of O(n log n)", "Maintains the relative order of equal elements", "Is easy to implement"]	Maintains the relative order of equal elements
4	Initial	Which sorting algorithm is known for its "divide and conquer"	["Insertion Sort", "Merge Sort", "Selection Sort", "Bubble Sort"]	Merge Sort

		strategy?		
5	Initial	An "in-place" sorting algorithm is one that sorts the input array:	["Without using any extra space", "Using a small, constant amount of extra space", "By creating a copy of the array", "In reverse order"]	Using a small, constant amount of extra space
6	bubble-sort	What is the worst-case time complexity of Bubble Sort?	["O(n)", "O(n log n)", "O(n^2)", "O(1)"]	O(n^2)
7	bubble-sort	Bubble Sort is a stable sorting algorithm.	["True", "False"]	TRUE
8	bubble-sort	In Bubble Sort, after the first pass through the array, where is the largest element located?	["At the beginning", "In the middle", "At the end", "Its position is unknown"]	At the end
9	selection-sort	What is the primary operation in Selection Sort?	["Swapping adjacent elements", "Finding the minimum element in the unsorted part", "Partitioning the array", "Merging sorted subarrays"]	Finding the minimum element in the unsorted part
10	selection-sort	The time complexity of Selection Sort in all cases (best, average, worst) is:	["O(n)", "O(n log n)", "O(n^2)", "Depends on the input"]	O(n^2)
11	selection-sort	How many swaps are performed in the worst case for an array of n elements using Selection Sort?	["O(n)", "O(n log n)", "O(n^2)", "O(1)"]	O(n)
12	insertion-sort	Insertion Sort is most efficient for:	["Large, unsorted arrays", "Large, sorted arrays", "Small arrays or nearly sorted arrays", "Arrays with random elements"]	Small arrays or nearly sorted arrays
13	insertion-sort	What is the best-case time complexity of Insertion Sort?	["O(n)", "O(n log n)", "O(n^2)", "O(1)"]	O(n)
14	insertion-sort	Insertion Sort is an in-place sorting algorithm.	["True", "False"]	TRUE
15	merge-sort	What is the space	["O(1)", "O(log n)", "O(n)",	O(n)

		complexity of a typical Merge Sort implementation?	"O(n^2)"]	
16	merge-sort	Merge Sort's time complexity is consistently:	["O(n)", "O(n log n)", "O(n^2)", "O(1)"]	O(n log n)
17	merge-sort	The core of Merge Sort is a function that:	["Finds the minimum element", "Swaps two elements", "Merges two sorted arrays", "Selects a pivot"]	Merges two sorted arrays
18	quick-sort	What is the worst-case time complexity of Quick Sort?	["O(n)", "O(n log n)", "O(n^2)", "O(1)"]	O(n^2)
19	quick-sort	The worst case for Quick Sort occurs when:	["The array is already sorted", "The pivot is always the smallest or largest element", "The array contains duplicate elements", "Both A and B"]	Both A and B
20	quick-sort	Quick Sort is generally faster in practice than Merge Sort because:	["It has better cache performance and lower constant factors", "It is a stable sort", "It has a better worst-case complexity", "It is easier to implement"]	It has better cache performance and lower constant factors

Appendix B : Student Score Dataset

Sorting Arcade Score Analysis										
No.	User Name	initial mcq	bubble sort	selection sort	insertion sort	merge sort	quick sort	initial mcq avarage	final mcq avarage	individual improvement
1	Nisal Lakshan	18.00	8.00	5.00	9.00	10.00	9.00	0.72	0.82	10.00
2	dewmin595	15.00	8.00	6.00	10.00	10.00	10.00	0.60	0.88	28.00
3	chathura de silva	16.00	8.00	5.00	10.00	9.00	9.00	0.64	0.82	18.00
4	chamindu suriya bandara	15.00	8.00	8.00	10.00	7.00	8.00	0.60	0.82	22.00
5	charuka vbc	14.00	8.00	9.00	7.00	9.00	9.00	0.56	0.84	28.00
6	kaceesha m	17.00	10.00	8.00	7.00	8.00	8.00	0.68	0.82	14.00
7	shanilkkahero	20.00	9.00	9.00	10.00	8.00	7.00	0.80	0.86	6.00
8	randika	18.00	8.00	4.00	7.00	5.00	5.00	0.72	0.58	-14.00
9	dimuthu	17.00	10.00	8.00	7.00	10.00	8.00	0.68	0.86	18.00
10	sasini	18.00	8.00	7.00	8.00	7.00	8.00	0.72	0.76	4.00
11	devindatp	18.00	9.00	8.00	7.00	8.00	8.00	0.72	0.80	8.00
12	kusal	15.00	7.00	8.00	8.00	7.00	10.00	0.60	0.80	20.00
13	supun	15.00	10.00	8.00	7.00	9.00	9.00	0.60	0.86	26.00
14	ayeshmantha	16.00	8.00	9.00	9.00	8.00	8.00	0.64	0.84	20.00
15	kanishka jd	12.00	9.00	10.00	8.00	9.00	9.00	0.48	0.90	42.00
16	dulan	14.00	10.00	8.00	8.00	7.00	7.00	0.56	0.80	24.00
17	nipuna	14.00	8.00	9.00	7.00	9.00	9.00	0.56	0.84	28.00
18	dimuth	15.00	7.00	6.00	10.00	8.00	7.00	0.60	0.76	16.00
19	chamini	18.00	10.00	7.00	7.00	8.00	7.00	0.72	0.78	6.00
20	kavindu	17.00	8.00	8.00	5.00	7.00	7.00	0.68	0.70	2.00
					total initial mcq avarage=			**12.88**		
					Group initial avarage=			**64.40%**		
					total final mcq avarage=				**16.14**	
					group final mcq avarage=				**80.70%**	
						Group improvemet=			**16.30%**	

$$\text{Group Initial Avg(\%)} = \frac{\text{Sum of all Initial Averages}}{\text{Number of Students}} \times 100\%$$

$$\text{Group Final Avg(\%)} = \frac{\text{Sum of all Final Averages}}{\text{Number Of Students}} \times 100\%$$

$$\text{Group Improvement} = (\text{Group Final Average} - \text{Group Initial Average})\ \%$$

Appendix C : Meating Minutes

NSBM GREEN UNIVERSITY TOWN

Research Project - Individual
Student Progression Report
[Students Copy]

01. Student Name ..W.P.S. Sandaruwan.....
02. Index Number ..20807.....
03. Degree Program ..BSc (Honours) in computer Science.....
04. Supervisor Name ..Miss. Kavishka Rajapaksha.....
05. Project Title ..Game based Learning as a Tool for understanding Sorting Algorithms.

Meeting Number	Meeting 01	Meeting 02	Meeting 03	Meeting 04	Meeting 05	Meeting 06	Meeting 07
Date	2024/ NOV-13	2024/ NOV-24	2024/ DEC/24	2025 02/11	2025 09/01	2025 09/06	
Student Signature							
Supervisor Signature							

Meeting Number	Meeting 08	Meeting 09	Meeting 10	Meeting 11	Meeting 12	Meeting 13	Meeting 14
Date							
Student Signature							
Supervisor Signature							

NSBM GREEN UNIVERSITY TOWN

Final Year Project – Supervisory meeting minutes

Meeting No: 01 (online)

Date : 2024-11-13

Project Title : Game Based Learning as a Tool for understanding sorting algorithms.

Name of the Student : W.P.S. Sandaruwan

Student ID : 22807

Name of the Supervisor : Miss. Kavishka Rajapaksha

Items discussed:

- Discuss about the structure of reash proposal
- focus on developing a web based system

Items to be completed before the next supervisory meeting:

- need to prepare Research proposal
- Create gantt chart, get an idea about time line
- create onedrive public ~~document~~ folder and share the link with madam, & need to add my works in that folder.

11/25/2024

Supervisor (Signature & Date)

Instructions to the supervisor: **Do not sign** if the above boxes are blank.

NSBM GREEN UNIVERSITY TOWN

Final Year Project – Supervisory meeting minutes

Meeting No: 02

Date : 2024-11-24

Project Title : Game Based Learning as a Tool for understanding Sorting ~~Algorithm~~ Algorithms.

Name of the Student : W.P.S. Sandaruwan

Student ID : 22807

Name of the Supervisor : Ms. Kavishka Rajapaksha

Items discussed:

* Descuss about some changes ~~about~~ in the proposal.
* ~~add idea~~
* collect more data and ideas about the topic.

Items to be completed before the next supervisory meeting:

* need to gather knowladge about two sorting algorithms.
* modify few changes in the proposal
* ~~Start~~ try to start de
* Learn about the frontend framework.

11/25/2024

Supervisor (Signature & Date)

Instructions to the supervisor: **Do not sign** if the above boxes are blank.

UNIVERSITY OF PLYMOUTH | NSBM GREEN UNIVERSITY TOWN

Final Year Project – Supervisory meeting minutes

Meeting No: 03

Date : 2025-01-24

Project Title : Game based learning as a tool for understanding sorting algorithms

Name of the Student : W.P.S. Sandaruwan

Students ID : 22807

Name of the Supervisor : Ms. Kavishka Rajapaksha

Items discussed:

* ~~Review~~ Review about previous works
* confirm the project proposal is finalized.

Items to be completed before the next supervisory meeting:

* need to develop user interfaces
* gather more information regarding to topics.
* gather information for literature review and Introduction phase.

1/24/2025

Supervisor (Signature & Date)

Instructions to the supervisor: **Do not sign** if the above boxes are blank.

NSBM GREEN UNIVERSITY TOWN

Final Year Project – Supervisory meeting minutes

Meeting No: 04

Date : 2025-09-11

Project Title : Game based learning as a tool for understanding sorting algorithms

Name of the Student : W.P.S Bandaranwan

Student ID : 22607

Name of the Supervisor : Ms. Kavishka Rajapaksha

Items discussed:

* About the presentation.
* About the progress of works.
*

Items to be completed before the next supervisory meeting:

* need to write the literature review.
* need to create game interfaces.
* ~~develop~~ first develop one algorithm.
* Develop game levels for 1st basic algorithm. (Bubble / mearge)

R 2/11/2025

Supervisor (Signature & Date)

Instructions to the supervisor: **Do not sign** if the above boxes are blank.

UNIVERSITY OF PLYMOUTH | NSBM GREEN UNIVERSITY TOWN

Final Year Project – Supervisory meeting minutes

Meeting No: 05

Date : 2025-09-01

Project Title : Game based learning as a Tool for understanding Sorting Algorithms

Name of the Student : W.P.S. Sandaruwan

Students ID : 22807

Name of the Supervisor : Ms. Kavishka Rajapaksha

Items discussed:

- need to more develop user interfaces of the system
- few errors identified. need to address them.

Items to be completed before the next supervisory meeting:

- create more user friendly game mode - UI
- Resolve errors.

You must update the applicat according to my guidlines.

4/11/2025 next meeting

9/1/2025

Supervisor (Signature & Date)

Instructions to the supervisor: **Do not sign** if the above boxes are blank.

UNIVERSITY OF PLYMOUTH | NSBM GREEN UNIVERSITY TOWN

Final Year Project – Supervisory meeting minutes

Meeting No: 06

Date : 2025-09-08

Project Title : Game based learning as a Tool for understanding Sorting Algorithms

Name of the Student : W.P.S. Sandaruwan

Students ID : 22807

Name of the Supervisor : Ms. Kavishka Rajapaksha

Items discussed:

- Application UI/UX
- final Documentation part.

Items to be completed before the next supervisory meeting:

- Need to modify more UI/UX effects
- Need to edit final thesis document
- Add more details and more descus about Methodology.

K 9/8/2025

Supervisor (Signature & Date)

Instructions to the supervisor: **Do not sign** if the above boxes are blank.